

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Математическая логика и теория формальных языков

Отчёт о выполнении лабораторной работы №4

«Доработка компилятора языка MiLan»

Студент,

группа 5130201/30102

_____ Путията М. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
Постановка задачи	4
1 Математическое описание	5
1.1 Язык MiLan	5
1.2 Лексический анализатор языка Milan	5
1.3 Грамматика языка MiLan	12
1.4 Описание операторов	15
1.5 Грамматика в БНФ	16
1.6 Синтаксические диаграммы грамматики	18
2 Особенности реализации	24
2.1 Список токенов	24
2.2 Структуры данных	25
2.3 Лексический анализатор (<code>scanner.h</code>)	25
2.4 Синтаксический анализатор (<code>parser.cpp</code>)	28
2.5 Визуальная среда разработки (<code>compile_visual</code>)	36
3 Результаты работы	39
3.1 Среда разработки	39
3.2 Стандартные операторы	42
3.3 Цикл <code>for</code>	43
3.4 Булевы константы и операции над ними	44
3.5 Одномерные массивы и операции над ними	45
3.6 Двумерные массивы и операции над ними	46
3.7 Конструкция <code>switch ... case</code>	47
Заключение	49
Список использованной литературы	51
Приложение А. Исходный код метода <code>initTM</code>	52

Введение

Лабораторная работа №4 заключается в доработке компилятора и виртуальной машины языка MiLan. В рамках данного варианта реализована поддержка работы со строками.

В данной лабораторной работе необходимо доработать компилятор языка MiLan согласно заданному варианту.

Вариант 7: Оператор выбора

```
switch(<выражение>)
begin
    case <значение 1>: <список операторов> break;
    case <значение 2>: <список операторов> break;
    ...
    default: <список операторов>
endcase
```

Также дополнительно к основному варианту будут реализованы следующие доработки компилятора:

- Оператор цикла

```
for(<инициализация>; <условие>; <выражение>) do
    <список операторов>
od
```

- Булевы константы, переменные и операции (&, &&, |, || !).
- Одномерные массивы и операции над ними (заполнение, поиск элемента, длина, замена элемента, реверс, изменение размера, сортировка, взятие элемента по индексу, взятие среза).
- Двумерные массивы и операции над ними (заполнение, поиск элемента, длина, замена элемента, реверс, изменение размера, сортировка, превращение в вектор, транспонирование, взятие элемента по индексу, взятие среза)
- Поэлементные арифметические операции над одномерными и двумерными массивами.
- Все виды комментариев (однострочные и многострочные).
- Перемножение двумерных массивов по правилам Шимбелла – `shimb(a, b, 1/0)`

Постановка задачи

Для выполнения поставленной задачи необходимо:

1. Проанализировать существующие языки программирования (C++, Ruby, JavaScript), на которых будет основан синтаксис доработок компилятора.
2. Изучить работу булевых операций в Java.
3. Выбрать типы операторов для одномерных и двумерных массивов (мутирующие или иммутабельные).
4. Создать новую синтаксическую диаграмму для лексического анализатора.
5. Расширить формальную грамматику языка MiLan в БНФ-нотации.
6. Построить синтаксические диаграммы расширенного языка MiLan.
7. Реализовать поддержку новых типов данных и операторов в компиляторе.

1 Математическое описание

1.1 Язык MiLan

Язык Милан — простой учебный язык программирования, в котором поддерживаются только целочисленные константы и идентификаторы, операторы присваивания, арифметические действия и сравнения. Также данный язык позволяет использовать управляющие конструкции (`if ... then ... else ... fi`, `while ... do ... od`, `begin ... end`) и IO-команды (`read`, `write`).

Программа написанная на языке Milan пишется между ключевыми словами `begin` и `end` и состоит из последовательности операторов и управляющих конструкций. Все операторы отделяются друг от друга с помощью точки с запятой, а после последнего оператора в блоке она не ставится. При компиляции программы не учитывается регистр символов.

1.2 Лексический анализатор языка Milan

Лексический анализатор проводит лексический анализ (первый этап трансляции), выделяя лексемы входного языка — минимальные единицы языка, которые имеют смысл. В естественном языке лексемами являются слова, а в языке программирования — идентификаторы, служебные символы, константы, операторы и т.д..

Так как к компилятору будет применено большое количество доработок было решено переписать лексический анализатор с нуля, реализовав его с помощью конечного автомата. Данное решение позволяет увеличить скорость обработки исходного кода программы.

Лексический анализатор реализуется с помощью конечного автомата, преобразующего входную цепочку символов в последовательность пар $\langle \text{Тип лексемы}, \text{Значение} \rangle$.

Если заданы:

- $\Sigma = \{\backslash t, \backslash n, _ , (,) , [,] , \{ , \} , * , + , - , / , . , : , ; , = , ! , < , > , | , \& \} \cup \{0, \dots, 9\} \cup \{A, \dots, Z\} \cup \{a, \dots, z\}$ — входной алфавит;
- $T = \{ \text{Div, Id, Arith, LBracket, RBracket, LCurly, RCurly, LSquare, RSquare, Assign, Semicolon, Comma, Var, Logical, Cmp, Keyword, Func, Comment, MultiComment} \}$ — множество типов лексем.
- $D = \{ \text{begin, end, if, then, else, fi, while, do, od, for, switch, case, default, endcase, break, int, bool, true, false, read, write, find, len, reverse, resize, replace, sorted, tovector, trans, shimb, fill, :=, =, ;, ,, :, !, \&, \&\&, |, ||, =, !=, <, <=, >, >=, +, -, *, /, (,), \"{, }\", [,], } \} \cup D_{id} \cup D_{var}$ — множество допустимых значений лексем, где

$$- D_{id} = \left\{ s \in \Sigma_{id}^+ \left| \begin{array}{l} |s| \in [1, 63], \\ s[0] \in \{A, \dots, Z\} \cup \{a, \dots, z\}, \\ \forall i \in [1, 63] : s[i] \in \Sigma_{id} \end{array} \right. \right\},$$

$$\Sigma_{id} = \{A, \dots, Z\} \cup \{a, \dots, z\} \cup \{0, \dots, 9\}$$

$$- D_{const} = \left\{ s \in \Sigma_{const}^+ \mid \begin{array}{l} |s| \in [1, 10], \\ |s| > 1 \rightarrow s[0] \neq 0 \end{array} \right\}$$

$$\Sigma_{const} = \{0, \dots, 9\}$$

То лексический анализатор будет реализовывать отображение $\Sigma^* \longrightarrow (T \times D)^*$, где

- Σ^* – замыкание алфавита входного языка;
- $(T \times D)^*$ – множество пар $\langle \text{Тип лексемы}, \text{Значение} \rangle$.

Для представления поведения конечного автомата (в данном случае лексического анализатора) можно использовать синтаксическую диаграмму, получаемую с помощью преобразования КА. Далее на рисунках 1 - 3 представлена синтаксическая диаграмма для лексического анализатора, разработанного для языка MiLan.

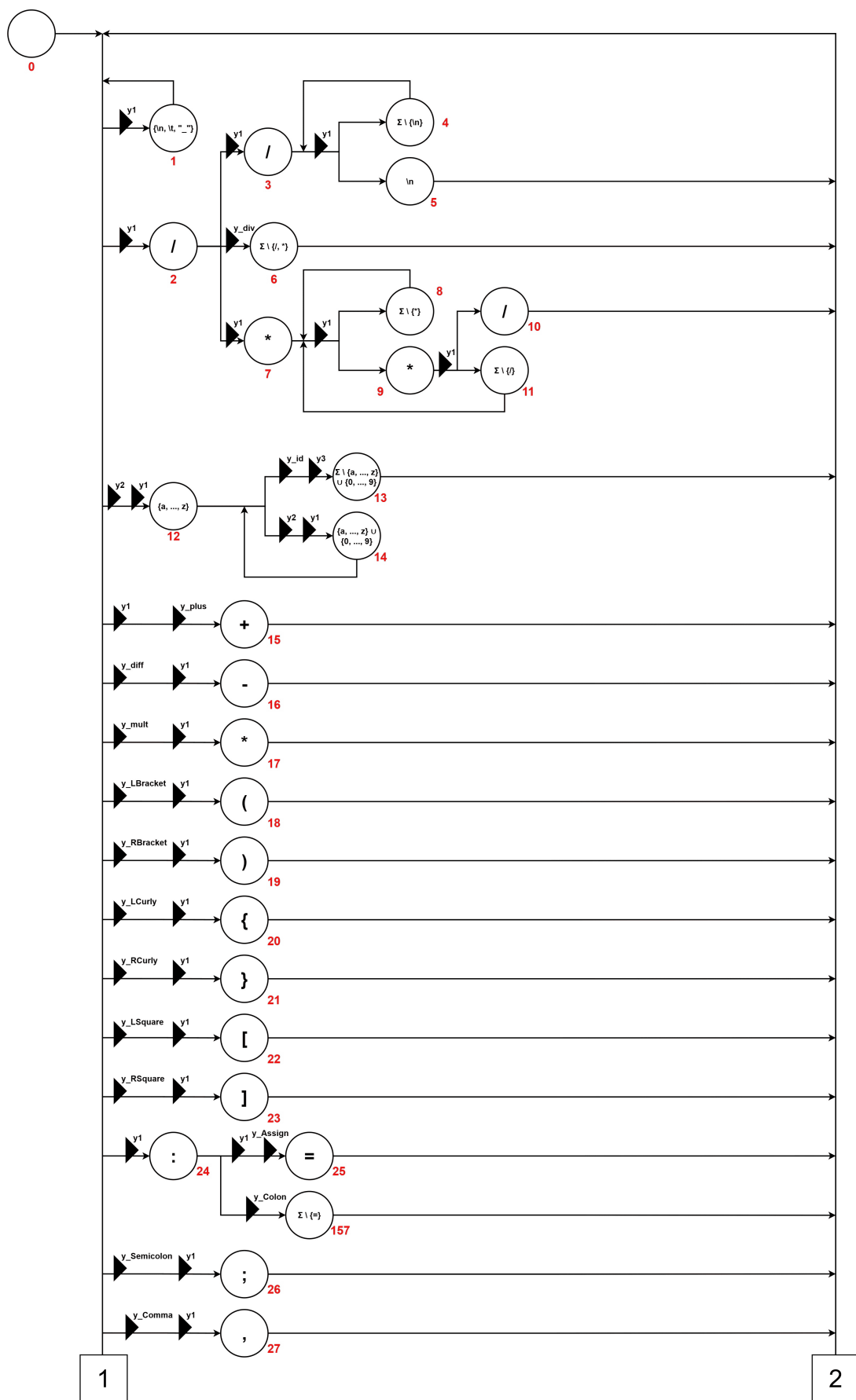


Рис. 1. Синтаксическая диаграмма лексического анализатора. Часть 1

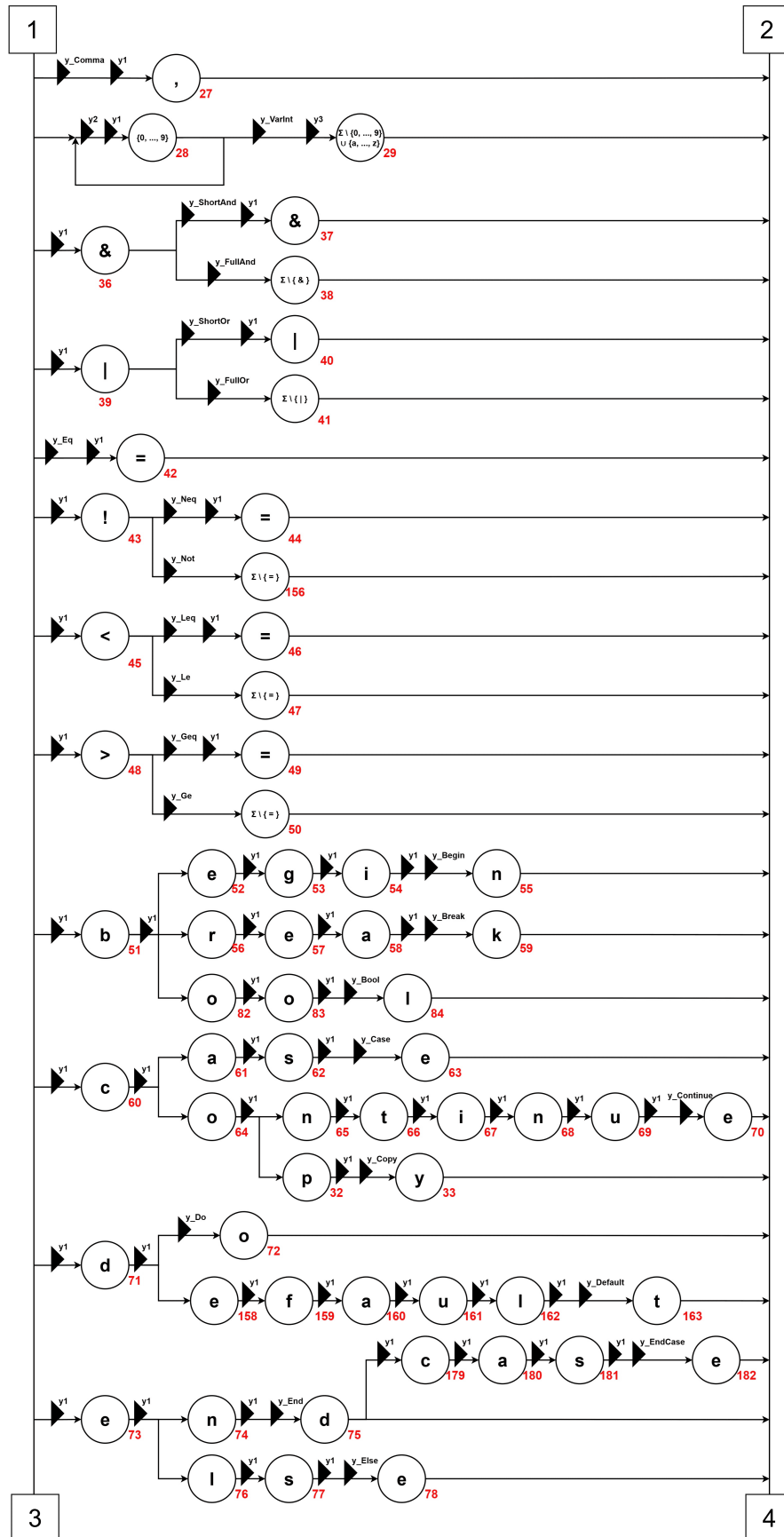


Рис. 2. Синтаксическая диаграмма лексического анализатора. Часть 2

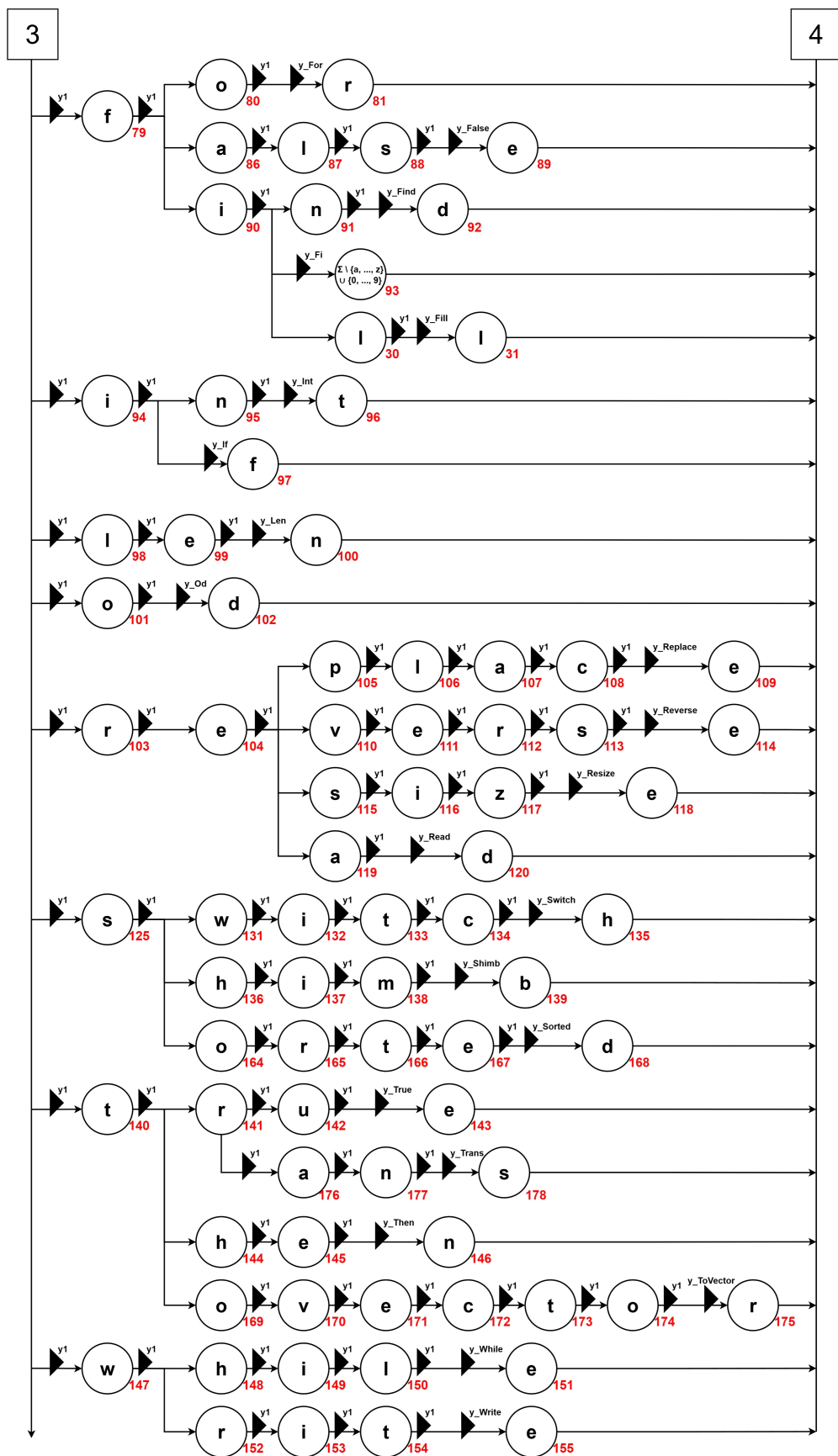


Рис. 3. Синтаксическая диаграмма лексического анализатора. Часть 3

Действия используемые в синтаксической диаграмме:

- $y1$ – сдвиг каретки;
- $y2$ – запись текущего символа в буфер;
- $y3$ – очистка буфера;
- y_Div – выдать лексему $\langle arith, / \rangle$;
- y_Id – выдать лексему $\langle Id, значение \rangle$;
- y_Plus – выдать лексему $\langle arith, + \rangle$;
- y_Diff – выдать лексему $\langle arith, - \rangle$;
- y_Mult – выдать лексему $\langle arith, * \rangle$;
- $y_LBracket$ – выдать лексему $\langle LBracket, (\rangle$;
- $y_RBracket$ – выдать лексему $\langle RBracket,) \rangle$;
- y_LCurly – выдать лексему $\langle LCurly, \{ \rangle$;
- y_RCurly – выдать лексему $\langle RCurly, \} \rangle$;
- $y_LSquare$ – выдать лексему $\langle LSquare, [\rangle$;
- $y_RSquare$ – выдать лексему $\langle RSquare,] \rangle$;
- y_Assign – выдать лексему $\langle Assign, := \rangle$;
- $y_Semicolon$ – выдать лексему $\langle Semicolon, ; \rangle$;
- y_Comma – выдать лексему $\langle Comma, , \rangle$;
- y_Var – выдать лексему $\langle Keyword, var \rangle$;
- $y_ShortAnd$ – выдать лексему $\langle Logic, \&\& \rangle$;
- $y_FullAnd$ – выдать лексему $\langle Logic, \& \rangle$;
- $y_ShortOr$ – выдать лексему $\langle Logic, || \rangle$;
- y_FullOr – выдать лексему $\langle Logic, | \rangle$;
- y_Equal – выдать лексему $\langle Cmp, = \rangle$;
- $y_NotEqual$ – выдать лексему $\langle Cmp, != \rangle$;
- y_Not – выдать лексему $\langle Logic, not \rangle$;
- $y_LessEqual$ – выдать лексему $\langle Cmp, <= \rangle$;
- y_Less – выдать лексему $\langle Cmp, < \rangle$;
- $y_GreaterEqual$ – выдать лексему $\langle Cmp, >= \rangle$;
- $y_Greater$ – выдать лексему $\langle Cmp, > \rangle$;

- y_Begin – выдать лексему <Keyword, begin>;
- y_Break – выдать лексему <Keyword, break>;
- y_Bool – выдать лексему <Keyword, bool>;
- y_Case – выдать лексему <Keyword, case>;
- y_Colon – выдать лексему <Colon, :>;
- y_Default – выдать лексему <Keyword, default>;
- y_Do – выдать лексему <Keyword, do>;
- y_End – выдать лексему <Keyword, end>;
- y_EndCase – выдать лексему <Keyword, endcase>;
- y_Else – выдать лексему <Keyword, else>;
- y_For – выдать лексему <Keyword, for>;
- y_False – выдать лексему <Var, 0>;
- y_Find – выдать лексему <Func, find>;
- y_Fi – выдать лексему <Keyword, fi>;
- y_Int – выдать лексему <Keyword, int>;
- y_If – выдать лексему <Keyword, if>;
- y_Len – выдать лексему <Func, len>;
- y_Od – выдать лексему <Keyword, od>;
- y_Replace – выдать лексему <Func, replace>;
- y_Reverse – выдать лексему <Func, reverse>;
- y_Resize – выдать лексему <Func, resize>;
- y_Read – выдать лексему <Keyword, read>;
- y_Switch – выдать лексему <Keyword, switch>;
- y_Shimb – выдать лексему <Func, Shimb>;
- y_Sorted – выдать лексему <Func, sorted>;
- y_True – выдать лексему <Var, 1>;
- y_Then – выдать лексему <Keyword, then>;
- y_ToVector – выдать лексему <Func, tovector>;
- y_Trans – выдать лексему <Func, trans>;
- y_While – выдать лексему <Keyword, while>;

- `y_Write` – выдать лексему `<Keyword, write>`;
- `y_Comment` – выдать лексему `<Comment, значение>`;
- `y_MultiComment` – выдать лексему `<MultiComment, значение>`;
- `y_Fill` – выдать лексему `<Func, fill>`;

1.3 Грамматика языка MiLan

Синтаксис языка MiLan определяется контекстно-свободной грамматикой, которая в иерархии Хомского относится к типу 2. Её ключевое свойство заключается в том, что левая часть любого правила вывода содержит ровно один нетерминальный символ (правила вывода имеют вид $A \rightarrow \gamma$). Все синтаксические конструкции MiLan подчиняются этому условию, поэтому его грамматика является контекстно-свободной.

Для разбора последовательностей лексем используется нисходящий синтаксический анализатор, реализованный методом рекурсивного спуска. Данный подход требует, чтобы грамматика обладала свойством LL(1): это позволяет строить для каждого нетерминала однозначные синтаксические диаграммы, где выбор следующей ветви определяется детерминированно по одной следующей лексеме. Свойство LL(1) гарантирует, что для корректного применения правила вывода достаточно знать текущий контекст анализа и первый ещё не обработанный терминал входной строки.

Контекстно-свободная грамматика задаётся как четверка объектов $G = (T, N, S, R)$, где:

- T — конечное множество терминальных символов: $T = \{\text{begin, end, if, then, else, fi, while, do, od, for, switch, case, default, endcase, break, int, bool, true, false, read, write, find, len, reverse, resize, replace, sorted, tovector, trans, shimb, fill, :=, =, ;, ,, :, !, \&, \&\&, |, ||, =, !=, <, <=, >, >=, +, -, *, /, (,), \{, \}, [,], identifier, integer}\}$.
- N — конечное множество нетерминальных символов: $N = \{\langle \text{Program} \rangle, \langle \text{DeclList} \rangle, \langle \text{Decl} \rangle, \langle \text{Type} \rangle, \langle \text{IdentList} \rangle, \langle \text{IdentInit} \rangle, \langle \text{ArrayInit} \rangle, \langle \text{MatrixInit} \rangle, \langle \text{MatrixRow} \rangle, \langle \text{StmtList} \rangle, \langle \text{Stmt} \rangle, \langle \text{AssignStmt} \rangle, \langle \text{IfStmt} \rangle, \langle \text{WhileStmt} \rangle, \langle \text{ForStmt} \rangle, \langle \text{SwitchStmt} \rangle, \langle \text{CaseList} \rangle, \langle \text{Case} \rangle, \langle \text{DefaultOpt} \rangle, \langle \text{ForInit} \rangle, \langle \text{ForStep} \rangle, \langle \text{WriteArg} \rangle, \langle \text{Expr} \rangle, \langle \text{BoolExpr} \rangle, \langle \text{BoolTerm} \rangle, \langle \text{BoolFactor} \rangle, \langle \text{BoolPrimary} \rangle, \langle \text{ArithExpr} \rangle, \langle \text{Term} \rangle, \langle \text{Factor} \rangle, \langle \text{Primary} \rangle, \langle \text{CmpOp} \rangle, \langle \text{ArrayExpr} \rangle, \langle \text{ArrayFuncExpr} \rangle, \langle \text{IndexExpr} \rangle, \langle \text{SliceExpr} \rangle, \langle \text{ArgList} \rangle\}$.
- $S \in N$ — начальный нетерминал: $S = \langle \text{program} \rangle$.
- R — конечное множество правил вывода вида $A \rightarrow \gamma$, где $A \in N$, $\gamma \in (T \cup N)^*$. Множество R задаётся следующими правилами вывода (где $|$ обозначает альтернативу, а $[\cdot]$ — необязательный элемент и $\{\cdot\}$ — повторение 0 или более раз):
 - $\langle \text{program} \rangle \rightarrow \text{begin } \langle \text{DeclSeq} \rangle \langle \text{StmtSeq} \rangle \text{end}$
 - $\langle \text{DeclSeq} \rangle \rightarrow \langle \text{type} \rangle \langle \text{ident-list} \rangle ; \langle \text{DeclSeq} \rangle \mid \varepsilon$

- $\langle \text{StmtSeq} \rangle \rightarrow \langle \text{stmt} \rangle ; \langle \text{StmtSeq} \rangle \mid \langle \text{stmt} \rangle \langle \text{StmtSeq} \rangle \mid \varepsilon$
- $\langle \text{type} \rangle \rightarrow \text{int} \mid \text{bool}$
- $\langle \text{ident-list} \rangle \rightarrow \langle \text{ident-init} \rangle \mid \langle \text{ident-init} \rangle , \langle \text{ident-list} \rangle$
- $\langle \text{ident-init} \rangle \rightarrow \langle \text{id} \rangle := \langle \text{expr} \rangle$
 - | $\langle \text{id} \rangle$
 - | $\langle \text{id} \rangle [\langle \text{integer} \rangle] := \langle \text{array-expr} \rangle$
 - | $\langle \text{id} \rangle [\langle \text{integer} \rangle] = \langle \text{array-expr} \rangle$
 - | $\langle \text{id} \rangle [\langle \text{integer} \rangle]$
 - | $\langle \text{id} \rangle [\langle \text{integer} \rangle] [\langle \text{integer} \rangle] := \langle \text{array-expr} \rangle$
 - | $\langle \text{id} \rangle [\langle \text{integer} \rangle] [\langle \text{integer} \rangle] = \langle \text{array-expr} \rangle$
 - | $\langle \text{id} \rangle [\langle \text{integer} \rangle] [\langle \text{integer} \rangle]$
- $\langle \text{stmt} \rangle \langle \text{id} \rangle \langle \text{assign-tail} \rangle$
 - | $\text{if } \langle \text{expr} \rangle \text{ then } \langle \text{StmtSeq} \rangle \text{ fi}$
 - | $\text{if } \langle \text{expr} \rangle \text{ then } \langle \text{StmtSeq} \rangle \text{ else } \langle \text{StmtSeq} \rangle \text{ fi}$
 - | $\text{while } \langle \text{expr} \rangle \text{ do } \langle \text{StmtSeq} \rangle \text{ od}$
 - | $\text{for } (\langle \text{ForInitOpt} \rangle ; \langle \text{expr} \rangle ; \langle \text{ForStepOpt} \rangle) \text{ do } \langle \text{StmtSeq} \rangle \text{ od}$
 - | $\text{switch } (\langle \text{expr} \rangle) \text{ begin } \langle \text{CaseSeq} \rangle \langle \text{DefaultOpt} \rangle \text{ endcase}$
 - | break
 - | $\text{write } (\langle \text{write-arg} \rangle)$
 - | $\text{read } (\langle \text{id} \rangle)$
 - | $\text{read } (\langle \text{id} \rangle [\langle \text{expr} \rangle])$
 - | $\text{read } (\langle \text{id} \rangle [\langle \text{expr} \rangle] [\langle \text{expr} \rangle])$
 - | $\langle \text{builtin-call} \rangle$
- $\langle \text{ForInitOpt} \rangle \rightarrow \langle \text{for-init} \rangle \mid \varepsilon$
- $\langle \text{ForStepOpt} \rangle \rightarrow \langle \text{for-step} \rangle \mid \varepsilon$
- $\langle \text{CaseSeq} \rangle \rightarrow \langle \text{Case} \rangle \langle \text{CaseSeq} \rangle \mid \varepsilon$
- $\langle \text{Case} \rangle \rightarrow \text{case } \langle \text{expr} \rangle : \langle \text{StmtSeq} \rangle \text{ break ;}$
 - | $\text{case } \langle \text{expr} \rangle : \langle \text{StmtSeq} \rangle$
- $\langle \text{DefaultOpt} \rangle \rightarrow \text{default} : \langle \text{StmtSeq} \rangle \mid \varepsilon$
- $\langle \text{assign-tail} \rangle \rightarrow := \langle \text{expr} \rangle$
 - | $:= \langle \text{array-expr} \rangle$
 - | $[\langle \text{expr} \rangle] := \langle \text{expr} \rangle$
 - | $[\langle \text{expr} \rangle] [\langle \text{expr} \rangle] := \langle \text{expr} \rangle$
 - | $[\langle \text{integer} \rangle] := \langle \text{array-expr} \rangle$
- $\langle \text{for-init} \rangle \rightarrow \langle \text{type} \rangle \langle \text{id} \rangle := \langle \text{expr} \rangle \mid \langle \text{id} \rangle := \langle \text{expr} \rangle$
- $\langle \text{for-step} \rangle \rightarrow \langle \text{id} \rangle := \langle \text{expr} \rangle$
- $\langle \text{write-arg} \rangle \rightarrow \langle \text{expr} \rangle$
 - | $\langle \text{array-expr} \rangle$
 - | $\langle \text{id} \rangle [\langle \text{integer} \rangle]$
 - | $\langle \text{id} \rangle [\langle \text{integer} \rangle : \langle \text{integer} \rangle]$
- $\langle \text{builtin-call} \rangle \rightarrow \text{find } (\langle \text{array-expr} \rangle , \langle \text{expr} \rangle)$
 - | $\text{find } (\langle \text{array-expr} \rangle , \langle \text{array-expr} \rangle)$
 - | $\text{len } (\langle \text{len-arg} \rangle)$

```

| reverse ( <array-expr> )
| sorted ( <array-expr> )
| sorted ( <array-expr> , <integer> )
| trans ( <array-expr> )
| tovector ( <id> )
| shimb ( <array-expr> , <array-expr> , <bool-val> )
| resize ( <array-expr> , <integer> , <expr> )
| resize ( <array-expr> , <integer> , <array-expr> )
| replace ( <array-expr> , <expr> , <expr> )
| replace ( <array-expr> , <expr> , <array-expr> )
| replace ( <array-expr> , <array-expr> , <expr> )
| replace ( <array-expr> , <array-expr> , <array-expr> )
| fill ( <id> , <expr> )
| fill ( <id> [ <integer> ] , <expr> )
- <expr> → <bool-term> | <bool-term> || <expr>
  | <bool-term> | <expr>
- <bool-term> → <bool-factor>
  | <bool-factor> && <bool-term>
  | <bool-factor> & <bool-term>
- <bool-factor> → ! <bool-primary> | <bool-primary>
- <bool-primary> → true | false
  | <arith-expr> <=> <arith-expr>
  | <arith-expr> != <arith-expr>
  | <arith-expr> < <arith-expr>
  | <arith-expr> <= <arith-expr>
  | <arith-expr> > <arith-expr>
  | <arith-expr> >= <arith-expr>
- <arith-expr> → <term>
  | <term> + <arith-expr>
  | <term> - <arith-expr>
- <term> → <factor>
  | <factor> * <term>
  | <factor> / <term>
- <factor> → - <primary>
  | ( int ) <primary>
  | <primary>
- <primary> → <integer> | true | false
  | <id>
  | <id> [ <expr> ]
  | <id> [ <expr> ] [ <expr> ]
  | <id> [ <integer> : <integer> ]
  | ( <expr> )
  | read ()
  | <builtin-call>
- <array-expr> → <elem-list>
  | <row-list>

```

```

| <id>
| <id> [ <integer> ]
| <id> [ <integer> : <integer> ]
| sorted ( <array-expr> )
| sorted ( <array-expr> , <integer> )
| trans ( <array-expr> )
| tovector ( <id> )
| shimb ( <array-expr> , <array-expr> , <bool-val> )
| resize ( <array-expr> , <integer> , <expr> )
| resize ( <array-expr> , <integer> , <array-expr> )
| replace ( <array-expr> , <expr> , <expr> )
| replace ( <array-expr> , <expr> , <array-expr> )
| replace ( <array-expr> , <array-expr> , <expr> )
| replace ( <array-expr> , <array-expr> , <array-expr> )

- <elem-list> → <expr>
| <expr> , <elem-list>

- <row-list> → <elem-list>
| <elem-list> , <row-list>

- <len-arg> → <id>
| <id> [ <expr> ]
| <id> [ <integer> : <integer> ]
| <array-expr>

- <bool-val> → 0
| 1
| true
| false

- <id> → <letter> <IdTail>
- <IdTail> → <letter> <IdTail>
| <digit> <IdTail>
| ε

- <integer> → <digit> <IntTail>
- <IntTail> → <digit> <IntTail>
| ε

- <digit> → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
- <letter> → a | b | ... | z
| A | B | ... | Z

```

1.4 Описание операторов

В течении выполнения данной лабораторной работы были добавлены следующие операторы:

- `len(array/matrix)` – возвращает длину заданного одномерного или двумерного массива.

- `replace(array/matrix, i, j)` – изменяет заданный одномерный или двумерный массив, заменяя все вхождения элемента i на элемент j .
- `reverse(array/matrix)` – изменяет заданный одномерный или двумерный массив, меняя порядок следования элементов на обратный.
- `resize(array/matrix, n, i)` – изменяет размер заданного одномерного или двумерного массива, если n больше текущей длины, то массив обрезается, иначе добавляется $n - l$ элементов i в конец массива, где l – длина массива.
- `sorted(array)` – сортирует элементы заданного одномерного массива.
- `sorted(matrix, i)` – сортируются элементы двумерного массива по значениям их i -ых элементов.
- `tovector(matrix)` – возвращает плоский вектор, образованный из элементов заданного двумерного массива.
- `trans(matrix)` – производит транспонирование заданного двумерного массива.
- `fill(array/matrix, i)` – заполняет заданный одномерный или двумерный массив элементами i .
- `find(array/matrix, i)` – возвращает позицию элемента i в заданном одномерном или двумерном массиве.
- `shimb(array1, array2, 1/0)` – возвращает двумерный массив, являющийся результатом перемножения заданных двумерных массивов по правилам Шимбелла.

1.5 Грамматика в БНФ

Расширенная грамматика языка MiLan с поддержкой новых конструкций и операторов приведена ниже.

```

1 <program> ::= "begin" {<type> <ident-list> ";"} {<stmt> [";"]} "end"
2
3 <type> ::= "int" | "bool"
4
5 <ident-list> ::= <ident-init> {",", <ident-init>}
6
7 <ident-init> ::= <id> [":=" <expr>]
8                | <id> "[" <integer> "]" [(":=") <array-expr>]
9                | <id> "[" <integer> "]" "[" <integer> "]" [(":=") <array-
10                  -expr>]
11
12 <stmt> ::= <id> <assign-tail>
13          | "if" <expr> "then" {<stmt> [";"]} ["else" {<stmt> [";"]} ] "fi"
14          | "while" <expr> "do" {<stmt> [";"]} "od"
15          | "for" "(" [<for-init>] ";" <expr> ";" [<for-step>] ")" "do" {
16            <stmt> [";"]} "od"
17          | "switch" "(" <expr> ")" "begin" {"case" <expr> ":" {<stmt>
18            [";"]} ["break"]} ["default" ":" {<stmt> [";"]} ] "endcase"
```

```

16         | "break"
17         | "write" "(" <write-arg> ")"
18         | "read" "(" <id> "[" <expr> "]" "[" <expr> "]" "]" ")"
19         | <builtin-call>
20
21 <assign-tail> ::= ":" <expr>
22             | ":" <array-expr>
23             | "[" <expr> "]" "[" <expr> "]" ":" <expr>
24             | "[" <integer> "]" ":" <array-expr>
25
26 <for-init> ::= [<type>] <id> ":" <expr>
27
28 <for-step> ::= <id> ":" <expr>
29
30 <write-arg> ::= <expr>
31             | <array-expr>
32             | <id> "[" <integer> "]"
33             | <id> "[" <integer> ":" <integer> "]"
34
35 <builtin-call> ::= "find" "(" <array-expr> "," (<expr> | <array-expr>)
36                ")"
37                | "len" "(" <len-arg> ")"
38                | "reverse" "(" <array-expr> ")"
39                | "sorted" "(" <array-expr> ["," <integer>] ")"
40                | "trans" "(" <array-expr> ")"
41                | "tovector" "(" <id> ")"
42                | "shimb" "(" <array-expr> "," <array-expr> "," <bool-
43                   val> ")"
44                | "resize" "(" <array-expr> "," <integer> "," (<expr> |
45                   <array-expr>) ")"
46                | "replace" "(" <array-expr> "," (<expr> | <array-expr>
47                   >) "," (<expr> | <array-expr>) ")"
48                | "fill" "(" <id> "[" <integer> "]" "]" "," <expr> ")"
49
50 <expr> ::= <bool-term> {("||" | "|") <bool-term>}
51
52 <bool-term> ::= <bool-factor> {("&&" | "&") <bool-factor>}
53
54 <bool-factor> ::= ["!"] <bool-primary>
55
56 <bool-primary> ::= "true" | "false" | <arith-expr> [("=" | "!=" | "<" |
57                "<=" | ">" | ">=") <arith-expr>]
58
59 <arith-expr> ::= <term> {("+" | "-") <term>}
60
61 <term> ::= <factor> {("*" | "/" ) <factor>}
62
63 <factor> ::= ["-"] <primary> | "(" "int" ")" <primary>
64
65 <primary> ::= <integer>
66             | "true" | "false"
67             | <id> ( "[" <expr> "]" "[" <expr> "]" "]" | "[" <integer>
68                   ":" <integer> "]" )
69             | "(" <expr> ")"
70             | "read" "(" ")"
71             | <builtin-call>

```

```

66
67 <array-expr> ::= "{" <elem-list> "}"
68               | "{" <row-list> "}"
69               | <id> ([ "[" <integer> "]" ] | [ "[" <integer> ":" <integer>
70                       > "]" ])
71               | "reverse" "(" <array-expr> ")"
72               | "sorted" "(" <array-expr> [ "," <integer> ] ")"
73               | "trans" "(" <array-expr> ")"
74               | "tovector" "(" <id> ")"
75               | "shimb" "(" <array-expr> "," <array-expr> "," <bool-val>
76                       > ")"
77               | "resize" "(" <array-expr> "," <integer> "," (<expr> | <
78                       array-expr>) ")"
79               | "replace" "(" <array-expr> "," (<expr> | <array-expr>)
80                       > "," (<expr> | <array-expr>) ")"
81
82 <elem-list> ::= <expr> { "," <expr> }
83
84 <row-list> ::= "{" <elem-list> "}" { "," { <elem-list> "}" }
85
86 <len-arg> ::= <id> ([ "[" <expr> "]" ] | [ "[" <integer> ":" <integer>
87                       > "]" ]) | <array-expr>
88
89 <bool-val> ::= "0" | "1" | "true" | "false"
90
91 <id> ::= (<letter> | "_" ) { <letter> | <digit> | "_" }
92
93 <integer> ::= <digit> { <digit> }
94
95 <digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
96
97 <letter> ::= "a" | ... | "z" | "A" | ... | "Z"

```

1.6 Синтаксические диаграммы грамматики

Далее на рисунках 4 - 6 представлены синтаксические диаграммы для расширенного языка MiLan.

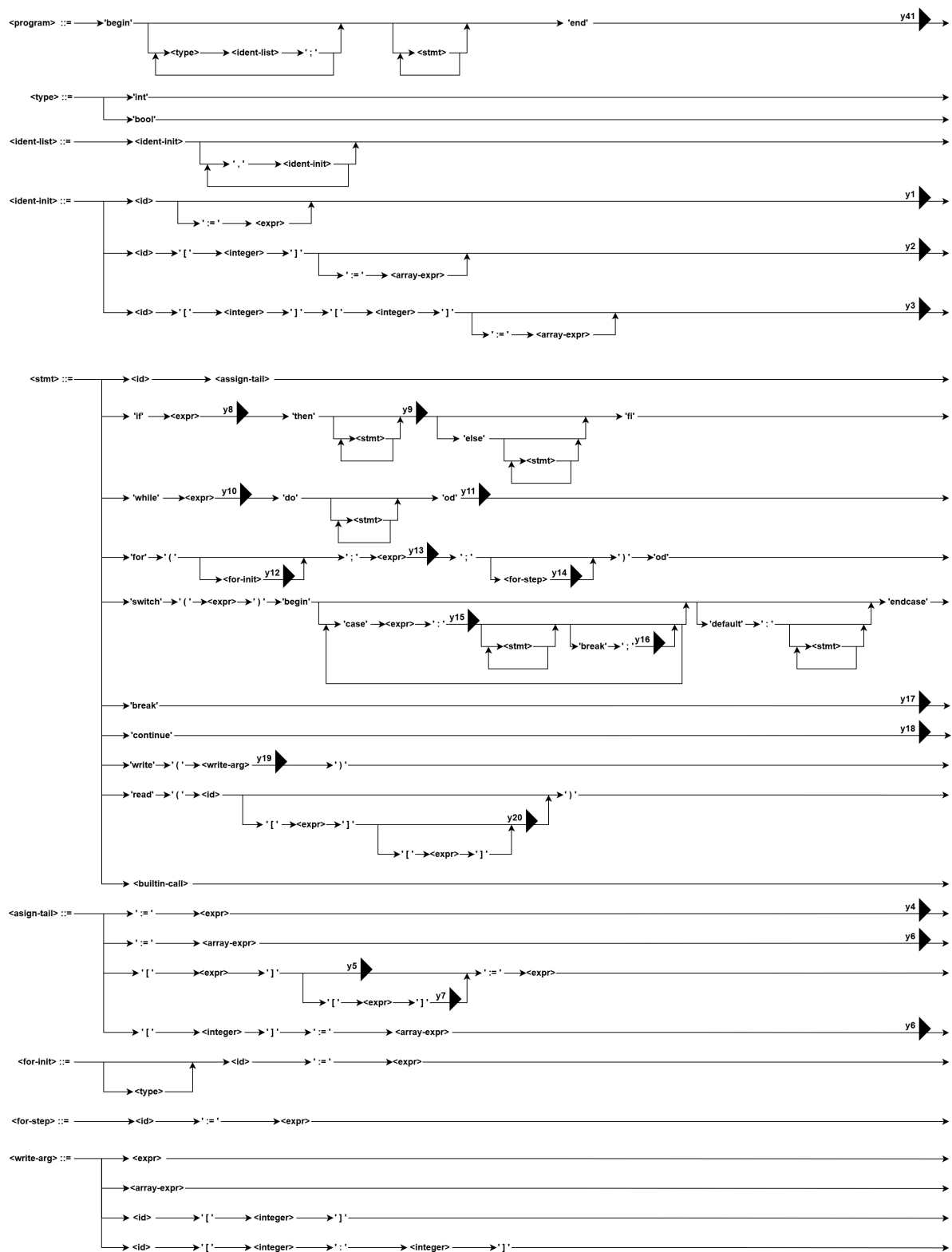


Рис. 4. Синтаксические диаграммы расш. ЯП MiLan. Часть 1

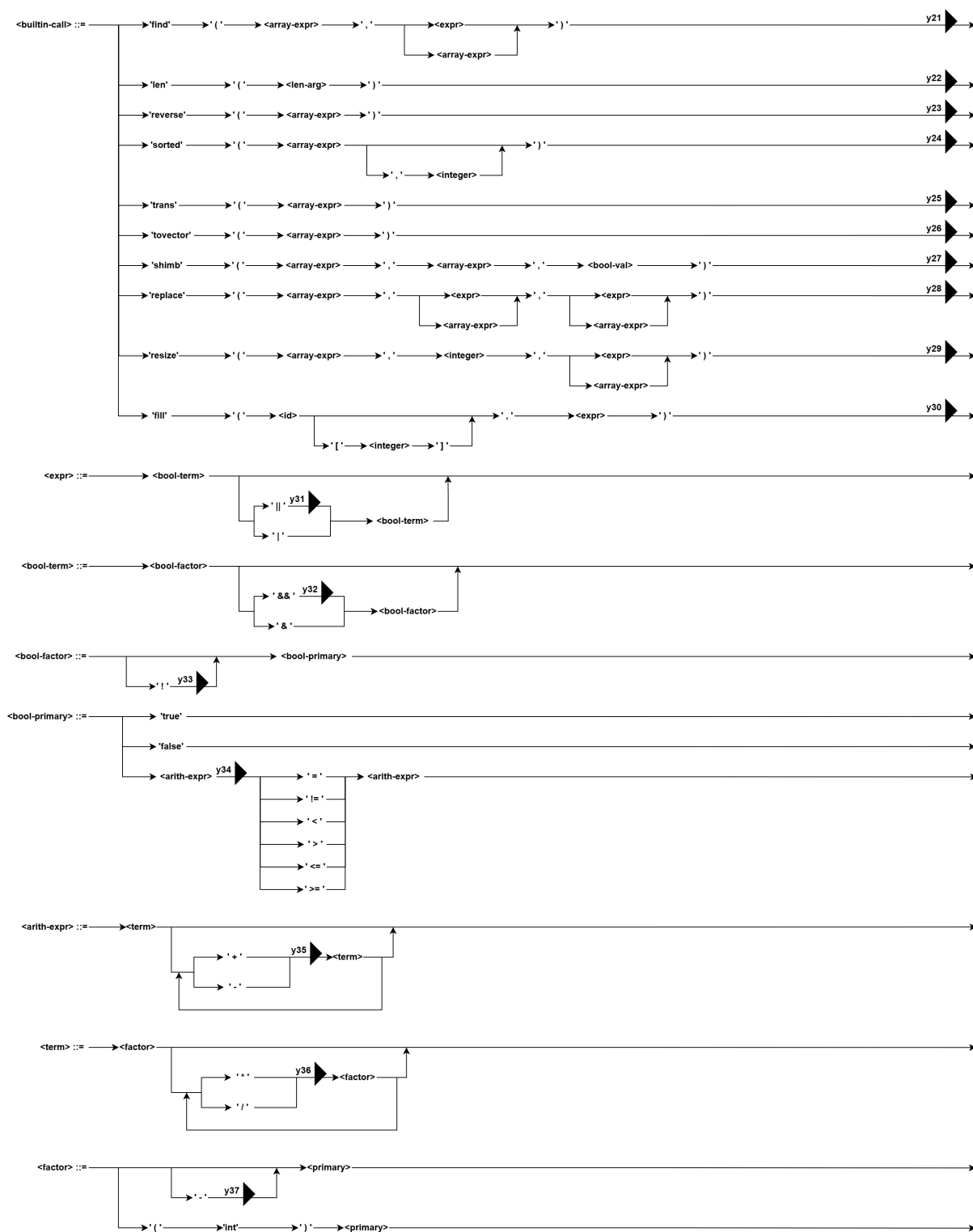


Рис. 5. Синтаксические диаграммы расш. ЯП MiLan. Часть 2

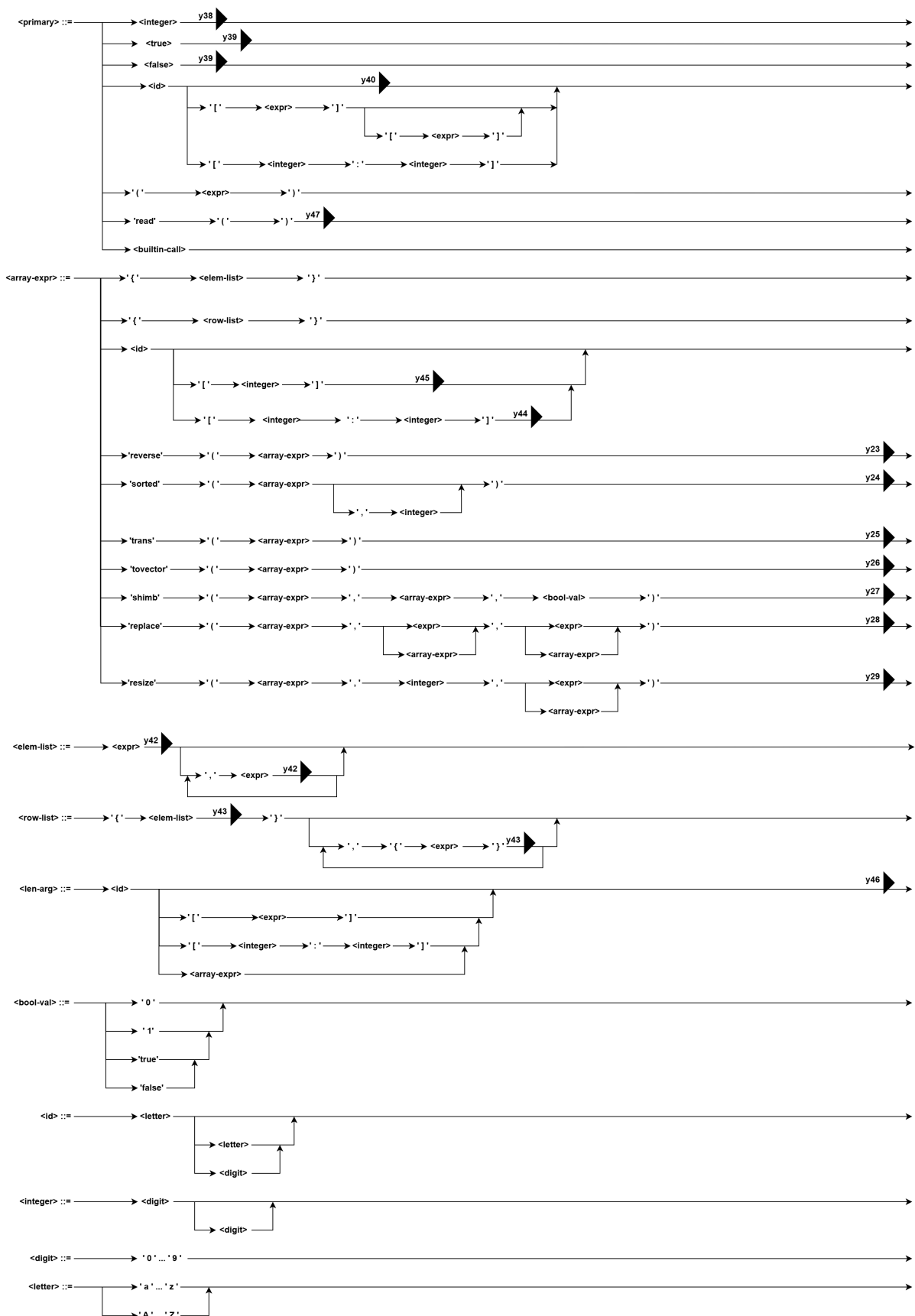


Рис. 6. Синтаксические диаграммы расш. ЯП MiLan. Часть 3

Описание выходов:

- y_1 — инициализация скаляра: STORE addr
- y_2 — инициализация массива: LOAD src_addr+i, STORE base+i
- y_3 — инициализация матрицы: LOAD src_addr+i, STORE base+i
- y_4 — присваивание скаляра: STORE vi_addr
- y_5 — присваивание элементу массива: PUSH cols, MULT, ADD, BSTORE vi_addr
- y_6 — присваивание строке матрицы: LOAD src_addr+c, STORE vi_addr+row*cols+c
- y_7 — присваивание элементу матрицы: PUSH cols, MULT, ADD, BSTORE vi_addr
- y_8 — условие if: JUMP_NO addr
- y_9 — конец then блока: JUMP addr_fi, JUMP_NO addr_else
- y_{10} — условие while: JUMP_NO addr_end
- y_{11} — конец тела while: JUMP addr_cond
- y_{12} — инициализация for: STORE var_addr
- y_{13} — условие for: JUMP_NO addr_out, JUMP addr_body
- y_{14} — шаг for: STORE var_addr, JUMP addr_cond
- y_{15} — case сравнение: DUP, COMPARE 0, JUMP_NO addr_next
- y_{16} — break в case: JUMP addr_endcase
- y_{17} — оператор break: JUMP addr_loop_end
- y_{18} — оператор continue: JUMP addr_continue
- y_{19} — вывод write: PRINT
- y_{20} — ввод read: INPUT, STORE vi_addr
- y_{21} — функция find (линейный поиск): LOAD i, PUSH len, COMPARE 2, JUMP_NO end, LOAD i, BLOAD addr, LOAD val, COMPARE 0, JUMP_YES found, LOAD i, PUSH 1, ADD, STORE i, JUMP loop, found: LOAD i, STORE res, end: LOAD res
- y_{22} — функция len: PUSH length_value
- y_{23} — функция reverse (обмен на месте): LOAD src[l], STORE tmp, LOAD src[r], STORE src[l], LOAD tmp, STORE src[r], PUSH 0
- y_{24} — функция sorted (сортировка пузырьком): LOAD a[j], LOAD a[j+1], COMPARE 3, JUMP_NO skip, LOAD a[j], STORE tmp, LOAD a[j+1], STORE a[j], LOAD tmp, STORE a[j+1], PUSH 0

- y_{25} — функция `trans` (транспонирование): `LOAD src[i][j], STORE tmp, LOAD src[j][i], STORE dst[i][j], LOAD tmp, STORE dst[j][i], PUSH res_addr`
- y_{26} — функция `tovector` (развёртка матрицы): `LOAD mat[r][c], STORE vec[i], PUSH vec_addr`
- y_{27} — функция `shimb` (min-plus умножение): `LOAD A[i][k], LOAD B[k][j], ADD, STORE acc, LOAD acc, LOAD cand, COMPARE 2, JUMP_NO skip, LOAD cand, STORE acc, STORE C[i][j], PUSH res_addr`
- y_{28} — функция `replace` (замена значений): `LOAD src[i], LOAD old_val, COMPARE 0, JUMP_NO skip, LOAD new_val, STORE src[i], PUSH 0`
- y_{29} — функция `resize` (изменение размера): `LOAD src[i], STORE dst[i], LOAD fill_val, STORE dst[new_i], PUSH 0`
- y_{30} — функция `fill` (заполнение): `LOAD fill_val, STORE dst[i], PUSH 0`
- y_{31} — логическое ИЛИ: `DUP, JUMP_NO addr, POP, ADD, COMPARE 3`
- y_{32} — логическое И: `DUP, JUMP_NO addr, POP, MULT, COMPARE 1`
- y_{33} — логическое НЕ: `INVERT, PUSH 1, ADD`
- y_{34} — сравнение: `COMPARE code (0-5)`
- y_{35} — сложение/вычитание: `ADD` или `SUB`
- y_{36} — умножение/деление: `MULT` или `DIV`
- y_{37} — унарный минус: `INVERT`
- y_{38} — константа: `PUSH value`
- y_{39} — булево: `PUSH 1` или `PUSH 0`
- y_{40} — переменная: `LOAD vi_addr`
- y_{41} — конец программы: `STOP`
- y_{42} — матричный литерал (строка): `STORE base+r*cols+c`
- y_{43} — массивный литерал: `STORE base+i`
- y_{44} — срез массива/матрицы: `LOAD src+i, STORE base+i`
- y_{45} — строка матрицы (копия): `LOAD mat+idx*cols+c, STORE temp+c`
- y_{46} — вычисление длины: `PUSH length`
- y_{47} — `read` внутри выражения: `INPUT`

2 Особенности реализации

Код компилятора состоит из нескольких компонентов: лексического анализатора (`scanner.h`, `scanner.cpp`), синтаксического анализатора (`parser.h`, `parser.cpp`) и визуальной среды разработки (`compile_visual.h`, `compile_visual.cpp`).

2.1 Список токенов

В лексическом анализаторе используется перечисление `LexemType`, содержащее следующий полный набор токенов:

Токен	Назначение
<code>Id</code>	Идентификатор переменной
<code>Var</code>	Целочисленный литерал
<code>Begin, End</code>	Ключевые слова <code>begin</code> , <code>end</code>
<code>If, Then, Else, Fi</code>	Условный оператор
<code>While, Do, Od</code>	Оператор цикла <code>while</code>
<code>For</code>	Оператор цикла <code>for</code>
<code>Switch, Case, Default, EndCase</code>	Оператор <code>switch</code>
<code>Break, Continue</code>	Выход из цикла и продолжение итерации
<code>Int, Bool</code>	Ключевые слова типов данных
<code>True, False</code>	Булевы литералы
<code>Write, Read</code>	Операторы вывода и ввода
<code>Find, Len, Reverse, Resize</code>	Встроенные функции массивов
<code>Replace, Sorted, Fill, Copy</code>	Встроенные функции массивов
<code>ToVector, Trans, Shimb</code>	Встроенные функции матриц
<code>Assign</code>	Оператор присваивания <code>:=</code>
<code>Plus, Diff, Mult, Div</code>	Арифметические операции
<code>Equal, NotEqual</code>	Операции сравнения <code>==</code> , <code>!=</code>
<code>Less, LessEqual, Greater, GreaterEqual</code>	Операции сравнения
<code>ShortAnd, FullAnd</code>	Логические операции <code>&&</code> , <code>&</code>
<code>ShortOr, FullOr</code>	Логические операции <code> </code> , <code> </code>
<code>Not</code>	Логическое отрицание <code>!</code>
<code>LBracket, RBracket</code>	Круглые скобки
<code>LCurly, RCurly</code>	Фигурные скобки
<code>LSquare, RSquare</code>	Квадратные скобки
<code>Semicolon, Comma, Colon</code>	Точка с запятой, запятая, двоеточие
<code>Comment, MultiComment</code>	Однострочный и многострочный комментарий

2.2 Структуры данных

Для хранения информации о переменных в таблице символов синтаксического анализатора введена структура `VarInfo`:

- `addr` — адрес переменной в памяти виртуальной машины;
- `type` — тип данных (`T_INT`, `T_BOOL`, `T_UNKNOWN`);
- `isArray`, `isMatrix` — признаки одномерного массива и матрицы;
- `arraySize`, `arraySize2` — размеры по первому и второму измерению соответственно.

Информация о контексте выполняемого цикла хранится во вложенной структуре `LoopInfo`:

- `continueAddr` — адрес инструкции, на которую передаётся управление при выполнении оператора `continue` (значение `-1` означает, что оператор находится внутри блока `switch`, где `continue` недопустим);
- `breakJumps` — список адресов зарезервированных инструкций `JUMP`.

2.3 Лексический анализатор (`scanner.h`)

Лексический анализатор реализован на основе конечного автомата, описанного шаблонным классом `FSM<Data>`.

Шаблонный класс `FSM<Data>`

Универсальный конечный автомат, параметризованный типом входных данных. Переходы между состояниями задаются предикатами (булевыми функциями над символом) и последовательностями действий, выполняемых при срабатывании перехода. Внутреннее представление таблицы переходов хранится в трёх векторах: `matrixConditions` (предикаты), `matrixActions` (действия), `matrixStates` (целевые состояния). Массив смещений `offsets` обеспечивает эффективный доступ к строке таблицы для текущего состояния без перебора всей таблицы.

Конструктор `FSM`.

Строит внутреннее представление таблицы переходов по карте переходов.

Вход: количество состояний автомата; карта переходов, где каждому состоянию сопоставлен список пар «целевое состояние, предикат и список действий»; обработчик ошибок, вызываемый при отсутствии подходящего перехода.

Выход: инициализированный объект автомата.

Код конструктора представлен на листинге 1.

```

1 FSM(int stateCount, TransitionMap transitions,
2     std::function<void(bool)> errorProcessor)
3     : errorProcessor(errorProcessor)
4 {
5     offsets.resize(stateCount + 1);
6     for (int state = 0; state < stateCount; ++state) {
7         offsets[state] = static_cast<int>(matrixActions.size());
8         auto it = transitions.find(state);
9         if (it != transitions.end()) {
10             for (const auto& [to, condAct] : it->second) {
11                 auto [condition, actions] = condAct;
12                 if (condition) {
13                     matrixActions.push_back(actions);
14                     matrixConditions.push_back(condition);
15                     matrixStates.push_back(to);
16                 }
17             }
18         }
19     }
20     offsets[stateCount] = static_cast<int>(matrixActions.size());
21 }

```

Метод doStep.

Выполняет один шаг автомата: извлекает первый символ из входной очереди, последовательно проверяет предикаты для текущего состояния и при первом срабатывании выполняет все связанные действия и переходит в целевое состояние.

Вход: карта переходов и символ из входной очереди.

Выход: совершённый переход с выполненными действиями, а также `true`, если очередь не пуста, и `false`, если входные данные закончились.

Код метода представлен на листинге 2.

```

1 bool doStep() {
2     int condStart = offsets[currentState];
3     int condEnd   = offsets[currentState + 1];
4     if (!dataProc.empty()) {
5         element = dataProc.front();
6         for (int i = condStart; i < condEnd; i++) {
7             if (matrixConditions[i](element)) {
8                 for (auto& action : matrixActions[i]) {
9                     if (action) action(element);
10                }
11                currentState = matrixStates[i];
12                return true;
13            }
14        }
15        errorProcessor(false);
16        return true;
17    }
18    errorProcessor(true);

```

```

19     return false;
20 }

```

Метод start.

Запускает автомат на входном тексте, последовательно вызывая `doStep` до исчерпания входной очереди символов.

Вход: входная последовательность символов (текст программы на языке MiLan), начальное состояние автомата.

Выход: таблица лексем и таблица ошибок, заполненные в процессе работы с помощью специальных действий при переходах; сброс в начальные состояния всех глобальных счётчиков автомата.

Код метода представлен на листинге 3.

Листинг 3. Метод FSM::start

```

1 void start(Data data, int startState = 0) {
2     if (startState < (int)offsets.size() - 1) {
3         dataProc.clear();
4         for (const ElementType& item : data) {
5             dataProc.push_back(item);
6         }
7         currentState = startState;
8     }
9     while (doStep()) {}
10    currentPos      = 1;
11    currentRow      = 1;
12    currentLexem    = 1;
13 }

```

Функция initTm

Конфигурирует конечный автомат, задавая полную таблицу переходов для распознавания всех токенов языка MiLan.

Вход: ссылки на буфер накопления символов текущей лексемы, текущую группу токенов, таблицу лексем, таблицу ошибок и таблицу соответствия лексем номерам строк исходного текста.

Выход: указатель на полностью инициализированный объект конечного автомата FSM.

Внутри функции определяются основные классы действий:

- `y1` — удаление обработанного символа из входной очереди;
- `y2` — добавление символа к буферу текущей лексемы;
- `y3` — очистка буфера лексемы;
- `y_Тип` — занесение лексемы соответствующего типа в таблицу токенов.

Из начального состояния 0 для каждого первого символа ключевого слова строится отдельная ветка распознавания. Например, все слова на букву R обрабатываются состояниями 103–120 и распознают `replace`, `reverse`, `resize`, `read`. Если начало очередного токена не совпадает ни с одним ключевым словом, он классифицируется как идентификатор.

Код данной функции представлен в Приложении А.

2.4 Синтаксический анализатор (`parser.cpp`)

Метод `parse`

Точка входа синтаксического анализатора. Запускает разбор программы и при отсутствии ошибок записывает сгенерированный код в выходной поток.

Вход: вектор указателей на лексемы и номера строк, на которых они расположены, полученный от лексического анализатора.

Выход: список ошибок в виде пар «текст сообщения, номер токена»; при успешном разборе — сгенерированный код в выходном потоке генератора кода.

Код метода представлен на листинге 4.

Листинг 4. Метод `Parser::parse`

```
1 std::vector<std::pair<std::string, int>>* Parser::parse()
2 {
3     program();
4     if (!error_) {
5         codegen_ -> flush();
6     }
7     return &errors_;
8 }
```

Метод `statementList`

Разбирает последовательность операторов в блоке. В отличие от оригинального анализатора языка MiLan, в расширенной версии точка с запятой требуется у всех операторов кроме последнего в блоке.

Вход: поток лексем начиная с текущей позиции; блок завершается одним из токенов: `end`, `od`, `fi`, `endcase`, `else`, `default`, `case`, `break`.

Выход: сгенерированные инструкции стек-машины для всех операторов блока; при нарушении правил постановки точки с запятой — соответствующие сообщения об ошибках.

Код метода представлен на листинге 5.

Листинг 5. Метод `Parser::statementList`

```
1 void Parser::statementList()
```

```

2 {
3     if (isBlockTerminator()) return;
4     while (true) {
5         if (see(Int) || see(Bool)) {
6             declaration();
7             mustBe(Semicolon, "Ожидалась';' после объявления");
8             if (isBlockTerminator()) break;
9             continue;
10        }
11        statement();
12        if (lastStmtWasCompound_) {
13            if (see(Semicolon)) {
14                reportError("Точка с запятой недопустима после"
15                    " составного оператора (if/while/for/switch)");
16                next();
17            }
18            if (isBlockTerminator()) break;
19        }
20        else if (see(Break)) {
21            mustBe(Semicolon,
22                "Ожидалась';' после оператора (перед 'break')");
23            break;
24        }
25        else if (isBlockTerminator()) {
26            if (see(Semicolon)) {
27                reportError("Лишняя';' перед"
28                    + closingTokenName(current().getType())
29                    + "': последний оператор в блоке не должен"
30                    " заканчиваться на';'");
31                next();
32            }
33            break;
34        }
35        else {
36            mustBe(Semicolon,
37                "Ожидалась';' после оператора (перед'"
38                + closingTokenName(current().getType()) + "')");
39        }
40    }
41 }

```

Метод isBlockTerminator

Определяет, является ли текущий токен признаком конца блока операторов.

Вход: текущий токен входного потока лексем.

Выход: `true`, если текущий токен завершает блок; иначе `false`.

Код метода представлен на листинге 6.

Листинг 6. Метод `Parser::isBlockTerminator`

```

1 bool Parser::isBlockTerminator() const
2 {
3     return see(End) || see(0d) || see(Fi) || see(EndCase)

```

```

4      || see(Else) || see(Default) || see(Case) || see(Break);
5  }

```

Метод identInit

Разбирает объявление одного идентификатора. Поддерживает три формы: скалярная переменная, одномерный массив и двумерный массив, каждая из них с необязательной инициализацией.

В качестве инициализатора принимается любой допустимый источник данных: константа, переменная, срез, результат встроенной функции.

Вход: тип данных объявляемой переменной (`int` или `bool`), поток лексем начиная с имени идентификатора.

Выход: запись в таблице символов с выделенным адресом в памяти; при наличии инициализатора также инструкции загрузки и записи начального значения.

Фрагмент кода представлен на листинге 7.

Листинг 7. Метод `Parser::identInit` (фрагмент)

```

1 void Parser::identInit(DataType type)
2 {
3     std::wstring name = current().getStr(); next();
4     if (match(LSquare)) {
5         int sz1 = std::stoi(current().getStr()); next();
6         mustBe(RSquare, "Ожидалась ']'");
7         if (match(LSquare)) {
8             int sz2 = std::stoi(current().getStr()); next();
9             mustBe(RSquare, "Ожидалась ']'");
10            int base = declareMatrix(name, type, sz1, sz2);
11            if (see(Equal) || see(Assign)) {
12                next();
13                VarInfo src = evalArrayExpr(type);
14                for (int i = 0; i < sz1 * sz2; ++i) {
15                    codegen_ -> emit(LOAD, src.addr + i);
16                    codegen_ -> emit(STORE, base + i);
17                }
18            }
19        } else {
20            int base = declareArray(name, type, sz1);
21            if (see(Equal) || see(Assign)) {
22                next();
23                VarInfo src = evalArrayExpr(type);
24                for (int i = 0; i < sz1; ++i) {
25                    codegen_ -> emit(LOAD, src.addr + i);
26                    codegen_ -> emit(STORE, base + i);
27                }
28            }
29        }
30    } else {
31        int addr = declareVar(name, type);
32        if (match(Assign)) {
33            expression();
34            codegen_ -> emit(STORE, addr);

```

```

35         }
36     }
37 }

```

Метод evalArrayExpr

Парсер источника массива, который принимает любое выражение, результатом которого является массив или матрица, и возвращает дескриптор блока памяти с уже записанными данными.

Вход: необязательный тип данных элементов; текущая позиция в потоке лексем, указывающая на начало одного из допустимых источников: инициализатор массива `{e,...}`, инициализатор матрицы `{{r},...}`, имя переменной, срез `arr[i:j]`, строка матрицы `mat[i]`, или одна из встроенных функций (`reverse`, `sorted`, `trans`, `shimb`, `resize`, `replace`, `tovector`).

Выход: объект `VarInfo` с адресом, размерами и признаком структуры (массив или матрица), данные которого уже записаны в памяти виртуальной машины.

Фрагмент кода представлен на листинге 8.

Листинг 8. Метод `Parser::evalArrayExpr` (фрагмент)

```

1 VarInfo Parser::evalArrayExpr(DataType type)
2 {
3     if (see(LCurly)) {
4         next();
5         if (see(LCurly)) { /* разбор literalной матрицы */ }
6         else                { /* разбор literalного массива */ }
7     }
8     if (match(ToVector)) return callToVectorImpl();
9     if (match(Copy))     return callCopyImpl();
10    if (match(Shimb))     return callShimbImpl();
11    if (match(Reverse))   return callReverseImpl();
12    if (match(Sorted))    return callSortedImpl();
13
14    std::wstring name = current().getStr(); next();
15    VarInfo& vi = getVar(name);
16    if (match(LSquare)) {
17        if (see(Var) && seeNext(Colon)) {
18            int from = std::stoi(current().getStr()); next(); next();
19            int to    = std::stoi(current().getStr()); next();
20            mustBe(RSquare, "Ожидалась_']'");
21            return sliceToTemp(vi, from, to);
22        } else {
23            int rowIdx = std::stoi(current().getStr()); next();
24            mustBe(RSquare, "Ожидалась_']'");
25            return matRowToTemp(vi, rowIdx);
26        }
27    }
28    return vi;
29 }

```

Иерархия разбора выражений

В оригинальном анализаторе метод `expression()` разбирал только арифметические выражения (+, −, *, /), а условия сравнения выделялись в отдельный метод `relation()`. В новой реализации метод `expression()` переименован в `arithmeticExpression()`, а попадание в него выполняется с помощью следующей последовательности вызовов:

`expression → boolExpression → boolTerm → boolFactor → boolPrimary →
→ arithmeticExpression → term → factor → primary`

Операторы `&&` и `||` реализованы с коротким замыканием: второй операнд не вычисляется, если результат уже определён первым. Операторы `&` и `|` всегда вычисляют оба операнда. Метод `boolPrimary` перенял функционал метода `relation()`, также добавив поддержку литералов `true` и `false`. Код метода `boolExpression` представлен на листинге 9.

Вход: текущая позиция в потоке лексем.

Выход: созданные инструкции для виртуальной машины для обработки логических выражений.

Листинг 9. Метод `Parser::boolExpression`

```
1 void Parser::boolExpression()
2 {
3     boolTerm();
4     while (see(FullOr) || see(ShortOr)) {
5         bool isShort = see(ShortOr);
6         next();
7         if (isShort) {
8             codegen_ -> emit(DUP);
9             int jR = codegen_ -> reserve();
10            int jE = codegen_ -> reserve();
11            codegen_ -> emitAt(jR, JUMP_NO, codegen_ -> getCurrentAddress());
12            ;
13            codegen_ -> emit(POP);
14            boolTerm();
15            codegen_ -> emit(PUSH, 0);
16            codegen_ -> emit(COMPARE, 1);
17            codegen_ -> emitAt(jE, JUMP, codegen_ -> getCurrentAddress());
18        } else {
19            boolTerm();
20            codegen_ -> emit(ADD);
21            codegen_ -> emit(PUSH, 0);
22            codegen_ -> emit(COMPARE, 3);
23        }
24    }
```

Методы ifStatement и whileStatement

Данные методы разбирают соответствующие операторы. Условие теперь разбирается через `expression()` вместо `relation()`, что позволяет использовать булевы операторы. В `whileStatement` добавлена поддержка `break` через стек для хранения контекстов.

Вход (для `ifStatement`): булево выражение-условие, блок операторов `then`, необязательный блок `else`, закрывающее слово `fi`.

Вход (для `whileStatement`): булево выражение-условие, блок операторов `while`, открывающее слово `do` и закрывающее слово `do`.

Выход: инструкции для генерации соответствующих блоков и инструкции `JUMP_NO` и `JUMP` для управления переходами при выходе из блоков.

Код методов представлен на листингах 10 и 11.

Листинг 10. Метод `Parser::ifStatement`

```
1 void Parser::ifStatement()
2 {
3     expression();
4     int jumpNoAddress = codegen_ -> reserve();
5     mustBe(Then, "Ожидалось 'then' после условия if");
6     statementList();
7     if (match(Else)) {
8         int jumpAddress = codegen_ -> reserve();
9         codegen_ -> emitAt(jumpNoAddress, JUMP_NO,
10                          codegen_ -> getCurrentAddress());
11         statementList();
12         codegen_ -> emitAt(jumpAddress, JUMP,
13                          codegen_ -> getCurrentAddress());
14     } else {
15         codegen_ -> emitAt(jumpNoAddress, JUMP_NO,
16                          codegen_ -> getCurrentAddress());
17     }
18     mustBe(Fi, "Ожидалось 'fi' для закрытия if");
19 }
```

Листинг 11. Метод `Parser::whileStatement`

```
1 void Parser::whileStatement()
2 {
3     int conditionAddress = codegen_ -> getCurrentAddress();
4     expression();
5     int jumpNoAddress = codegen_ -> reserve();
6     mustBe(Do, "Ожидалось 'do' после условия while");
7     LoopInfo li;
8     li.continueAddr = conditionAddress;
9     loops_.push(li);
10    statementList();
11    mustBe(Ord, "Ожидалось 'od' для закрытия while");
12    codegen_ -> emit(JUMP, conditionAddress);
13    int afterLoop = codegen_ -> getCurrentAddress();
14    codegen_ -> emitAt(jumpNoAddress, JUMP_NO, afterLoop);
15    for (int a : loops_.top().breakJumps)
```

```

16         codegen_ ->emitAt(a, JUMP, afterLoop);
17     loops_.pop();
18 }

```

Метод forStatement

Разбирает оператор цикла `for`. Поддерживает объявление переменной непосредственно в заголовке цикла.

Вход: заголовок цикла в скобках (инициализатор, условие, шаг, разделённые точками с запятой), ключевое слово `do`, тело цикла, ключевое слово `od`.

Выход: инструкции инициализации; инструкции проверки условия с переходами (`JUMP_NO` за цикл, `JUMP` к телу); инструкции шага; указание адресов для `break` и `continue`.

Фрагмент кода представлен на листинге 12.

Листинг 12. Метод `Parser::forStatement` (фрагмент)

```

1 void Parser::forStatement()
2 {
3     mustBe(LBracket, "Ожидалась_'('после_'for'");
4     /* разбор инициализатора */
5     mustBe(Semicolon, "Ожидалась_';'после_инициализатора_for");
6
7     int condAddr = codegen_ ->getCurrentAddress();
8     expression();
9     int jmpOut = codegen_ ->reserve();
10    int jmpBody = codegen_ ->reserve();
11    mustBe(Semicolon, "Ожидалась_';'после_условия_for");
12
13    int stepAddr = codegen_ ->getCurrentAddress();
14    /* разбор шага */
15    codegen_ ->emit(JUMP, condAddr);
16    mustBe(RBracket, "Ожидалась_')'_после_шага_for");
17    mustBe(Do, "Ожидалось_'do'_после_for(...)");
18
19    int bodyAddr = codegen_ ->getCurrentAddress();
20    codegen_ ->emitAt(jmpOut, JUMP_NO, 0);
21    codegen_ ->emitAt(jmpBody, JUMP, bodyAddr);
22
23    LoopInfo li; li.continueAddr = stepAddr; loops_.push(li);
24    statementList();
25    mustBe(Od, "Ожидалось_'od'_для_закрытия_for");
26
27    codegen_ ->emit(JUMP, stepAddr);
28    int afterLoop = codegen_ ->getCurrentAddress();
29    codegen_ ->emitAt(jmpOut, JUMP_NO, afterLoop);
30    for (int a : loops_.top().breakJumps)
31        codegen_ ->emitAt(a, JUMP, afterLoop);
32    loops_.pop();
33 }

```

Метод switchStatement

Разбирает оператор `switch`. Ключевое слово `break` и следующая за ним точка с запятой обязательны в конце каждой ветки `case`.

Вход: выражение для определения ветви в скобках, блок `begin / endcase` с произвольным числом веток `case` и необязательной веткой `default`.

Выход: для каждой ветки — инструкции `DUP`, вычисление метки, `COMPARE 0`, `JUMP_NO` на следующую проверку; после тела ветки — `JUMP` за весь оператор; в конце — `POP` для снятия значения переключателя со стека.

Код метода представлен на листинге 13.

Листинг 13. Метод `Parser::switchStatement`

```
1 void Parser::switchStatement()
2 {
3     mustBe(LBracket, "Ожидалась_'('после_'switch'");
4     expression();
5     mustBe(RBracket, "Ожидалась_')'после_выражения_switch");
6     mustBe(Begin, "Ожидалось_'begin'после_switch(...)");
7     LoopInfo swInfo; swInfo.continueAddr = -1; loops_.push(swInfo);
8     std::vector<int> breakToEnd;
9     while (see(Case)) {
10         next();
11         codegen_->emit(DUP);
12         expression();
13         codegen_->emit(COMPARE, 0);
14         int jmpNext = codegen_->reserve();
15         mustBe(Colon, "Ожидалось_':'после_значения_case");
16         statementList();
17         mustBe(Break, "Ожидалось_'break'в_конце_case");
18         mustBe(Semicolon, "Ожидалась_';'после_'break'");
19         breakToEnd.push_back(codegen_->reserve());
20         codegen_->emitAt(jmpNext, JUMP_NO,
21             codegen_->getCurrentAddress());
22     }
23     if (match(Default)) {
24         mustBe(Colon, "Ожидалось_':'после_'default'");
25         statementList();
26     }
27     mustBe(EndCase, "Ожидалось_'endcase'для_закрытия_switch");
28     codegen_->emit(POP);
29     int afterSwitch = codegen_->getCurrentAddress();
30     for (int a : breakToEnd)
31         codegen_->emitAt(a, JUMP, afterSwitch);
32     for (int a : loops_.top().breakJumps)
33         codegen_->emitAt(a, JUMP, afterSwitch);
34     loops_.pop();
35 }
```

2.5 Визуальная среда разработки (compile_visual)

В рамках выполнения лабораторной работы также была разработана среда разработки на базе фреймворка Qt, реализованная в классе `compile_visual`. Среда предоставляет следующие возможности:

- редактирование `.mil`-файлов с вкладками или работа с проектами (папка с несколькими `.mil`-файлами);
- просмотр сгенерированного кода в отдельных `.ms`-вкладках;
- лексический анализ в реальном времени с подсветкой синтаксиса и подчёркиванием ошибок;
- компиляция исходного текста в код для виртуальной машины;
- запуск виртуальной машины как дочернего процесса с интерактивным вводом.

Метод Build

Реализует процесс компиляции активного `.mil`-файла: лексический анализ, фильтрацию комментариев, синтаксический анализ и запись сгенерированного кода в `.ms`-файл.

Вход: содержимое активной вкладки редактора и путь к файлу на диске.

Выход: файл с расширением `.ms` содержащий инструкции для виртуальной машины; при наличии ошибок — список сообщений с номером строки и токена, отображаемый в панели ошибок.

Фрагмент кода представлен на листинге 14.

Листинг 14. Метод `compile_visual::Build` (фрагмент)

```
1 bool compile_visual::Build() {
2     QPlainTextEdit* milEditor =
3         qobject_cast<QPlainTextEdit*>(ui.tabs_mil->currentWidget());
4     if (!milEditor) return false;
5
6     std::wstring content =
7         milEditor->toPlainText().toStdWString() + L"\n";
8     result.clear();
9     fsm->start(content, 0);
10    if (!result.errors.empty()) return false;
11
12    auto it = result.lexems.begin();
13    while (it != result.lexems.end()) {
14        if ((*it)->getType() == Comment ||
15            (*it)->getType() == MultiComment)
16            it = result.lexems.erase(it);
17        else ++it;
18    }
19
20    std::stringstream codeStream;
21    CodeGen codegen(codeStream);
```

```

22     Parser p(result.lexems, &codegen);
23     auto* errors = p.parse();
24     if (!errors->empty()) {
25         for (const auto& error : *errors)
26             ui.errorInfo->appendPlainText(error.first.c_str());
27         return false;
28     }
29
30     QFile msFile(msPath);
31     msFile.open(QIODevice::WriteOnly | QIODevice::Text);
32     QTextStream out(&msFile);
33     out << QString::fromStdString(codeStream.str());
34     msFile.close();
35     return true;
36 }

```

Метод runLexicalAnalysis

Запускается по таймеру при изменении текста в редакторе и выполняет лексический анализ в реальном времени.

Вход: содержимое активной вкладки редактора .mil-файла.

Выход: обновлённая таблица лексем; список лексических ошибок в панели ошибок; цветовая подсветка токенов в редакторе.

Код метода представлен на листинге 15.

Листинг 15. Метод compile_visual::runLexicalAnalysis

```

1 void compile_visual::runLexicalAnalysis() {
2     QPlainTextEdit* editor =
3         qobject_cast<QPlainTextEdit*>(ui.tabs_mil->currentWidget());
4     if (!editor) return;
5
6     std::wstring content = editor->toPlainText().toStdWString();
7     result.clear();
8     fsm->start(content + L'\n', 0);
9
10    ui.errorInfo->clear();
11    if (!result.errors.empty()) {
12        for (Error* err : result.errors) {
13            ui.errorInfo->appendPlainText(
14                QString("Ошибка [%1:%2]: %3 (%\"%4\") ")
15                    .arg(err->getRow()).arg(err->getPos())
16                    .arg(QString::fromStdWString(err->getReason()))
17                    .arg(QString::fromStdWString(err->getText())));
18        }
19    }
20    applyHighlighting(editor, result);
21 }

```

Метод applyHighlighting

Применяет цветовую подсветку к тексту в редакторе на основе типов распознанных токенов, используя механизм `QTextEdit::ExtraSelection`.

Дополнительно подчёркивает волнистой линией красного цвета позиции лексических ошибок.

Вход: редактор `QPlainTextEdit` с текстом программы; результат лексического анализа, содержащий таблицу лексем и таблицу ошибок.

Выход: обновлённый список `ExtraSelection` редактора, выделяющий ключевые слова (оранжевый), встроенные функции (жёлтый), типы данных (фиолетовый), числа (зелёный), булевы значения (синий), операторы и идентификаторы (светло-серый), комментарии (серый), а также позиции ошибок (волнистое красное подчёркивание).

Метод sendProcessInput

Передаёт строку, введённую пользователем, в стандартный входной поток запущенного процесса виртуальной машины.

Вход: текст из поля ввода пользовательского интерфейса.

Выход: данные в стандартном входном потоке дочернего процесса; введённая строка дублируется в консольную панель; поле ввода очищается.

Код метода представлен на листинге 16.

Листинг 16. Метод `compile_visual::sendProcessInput`

```
1 void compile_visual::sendProcessInput() {
2     if (m_process && m_process->state() == QProcess::Running) {
3         QString input = ui.lineInput->text();
4         m_process->write((input + "\n").toLocal8Bit());
5         QTextCursor cursor = ui.console_out->textCursor();
6         cursor.movePosition(QTextCursor::End);
7         cursor.insertText(input);
8         ui.console_out->setTextCursor(cursor);
9         ui.console_out->ensureCursorVisible();
10        ui.lineInput->clear();
11    }
12 }
```

3 Результаты работы

3.1 Среда разработки

На рисунке 7 представлен внешний вид созданной среды разработки сразу после её запуска, вдохновленный дизайном Visual Studio. Среда разработки представляет пользователю 5 текстовых окон: редактор исходного кода (слева сверху), редактор Р-кода (справа сверху), вывод результатов компиляции (слева снизу), вывод списка ошибок (слева снизу) и консоль для взаимодействия с запускаемой программой (справа снизу).

Размер окон может быть изменён пользователем при помощи перетягивания их краёв, при этом соседние окна будут сдвигаться так чтобы всегда было заполнено всё пространство среды разработки, а также может их полностью скрывать с помощью перетягивании к краю окна приложения. Редактирование текста разрешено только в двух верхних окнах, а во всех остальных доступен только просмотр их содержимого, а ввод данных в программу производится с помощью специальной строки ввода, расположенной под консолью.

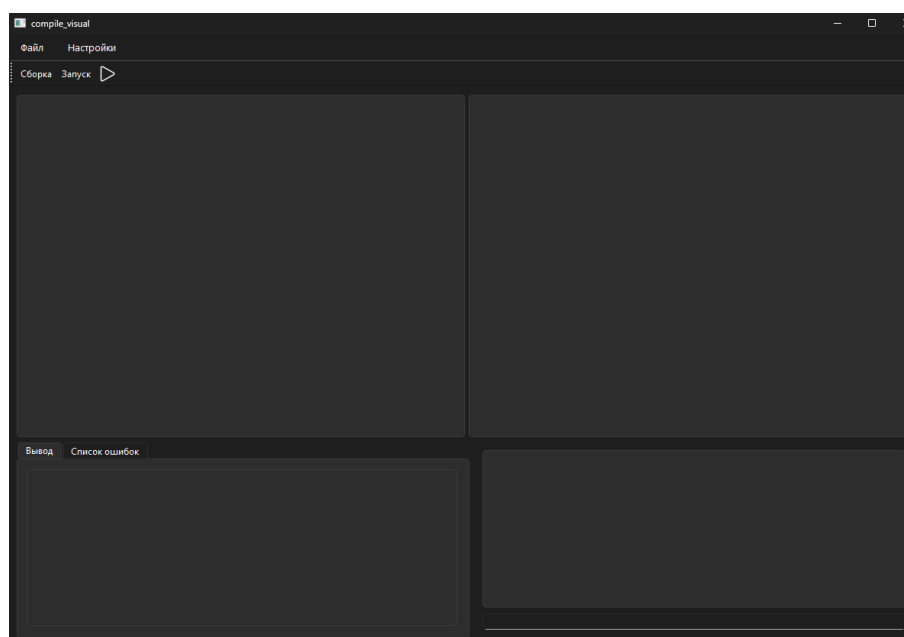
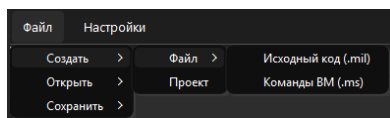
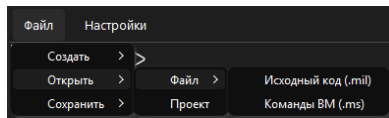


Рис. 7. Визуальная среда разработки сразу после запуска

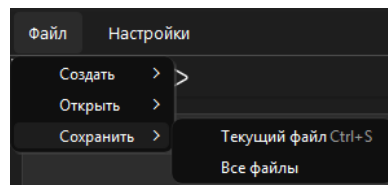
Создание, открытие и сохранение файлов может выполняться пользователем через верхний набор меню «Файл», а настройка автоподсветки лексем — через меню «Настройки». Иерархия меню «Файл» представлена на рисунках 8(а) - 8(с).



(a) Подменю «Создать»



(b) Подменю «Открыть»



(c) Подменю «Сохранить»

Рис. 8. Иерархия меню «Файл»

После открытия файла, сборки программы и её запуска все текстовые окна заполняются выводами. На рисунке 9 изображена среда разработки после выполнения данных действий.

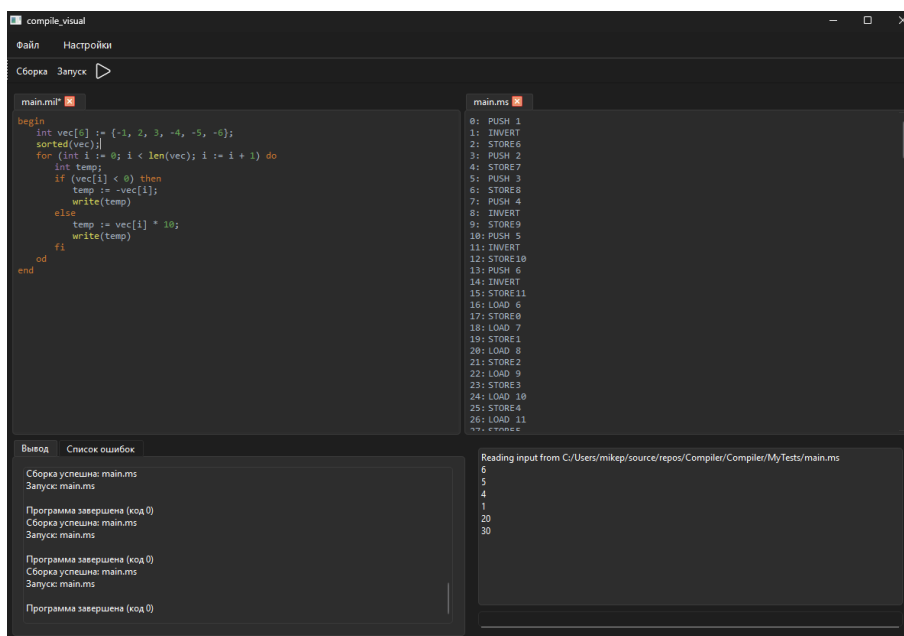


Рис. 9. Визуальная среда разработки после открытия файла, сборки и запуска программы

Через секунду после остановки ввода текста пользователем в редактор исходного кода запускается лексический анализ, который позволяет создать подсветку лексем и ошибок. В случае обнаружения лексической ошибки, место её обнаружения подчеркивается красным цветом. Если во время компиляции программы были обнаружены такие ошибки, пишется соответствующее сообщение в вывод компиляции, при этом прошлый сгенерированный код остаётся доступен для запуска.

На рисунке 10 изображен вывод компиляции и подчеркивание ошибок при наличии в коде программы идентификаторов начинающихся с цифры и состоящих из символов кириллицы.

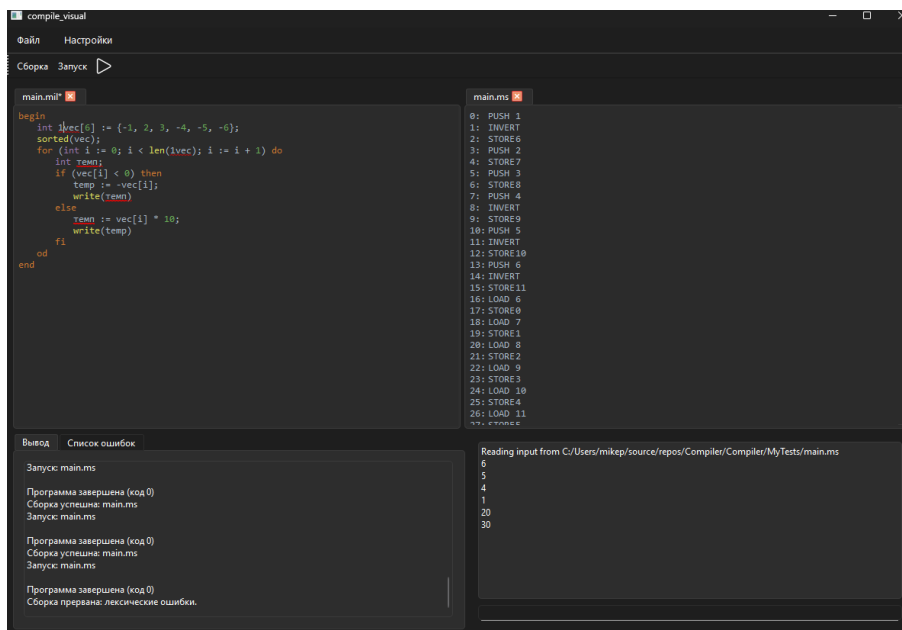


Рис. 10. Визуальная среда разработки после сборки программы, содержащей лексические ошибки

Также если во время компиляции программы во время синтаксического анализа обнаружены синтаксические ошибки, компиляция завершается и в её вывод пишутся сообщения об ошибках. На рисунке 11 представлен вывод компиляции при наличии необъявленного идентификатора и точки с запятой после последнего оператора в блоке.

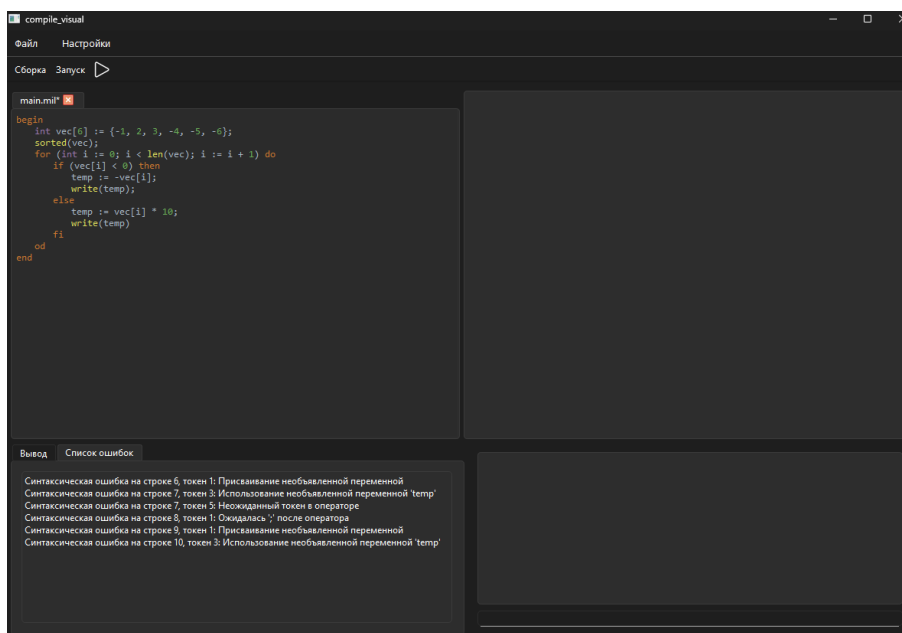


Рис. 11. Визуальная среда разработки после сборки программы, содержащей синтаксические ошибки

Среда разработки поддерживает работу сразу с несколькими файлами, при этом

сценарии работы текущий файл для каждого окна можно переключать с помощью нажатия на соответствующую вкладку над редактором. При изменении файла к его названию во вкладке добавляется символ «*», а при попытке выхода из программы с одним или несколькими несохранёнными файлами выводится диалог с предложением сохранения для каждого такого файла.

На рисунке 12 изображена попытка закрытия среды разработки при налии нескольких несохранённых файлов.

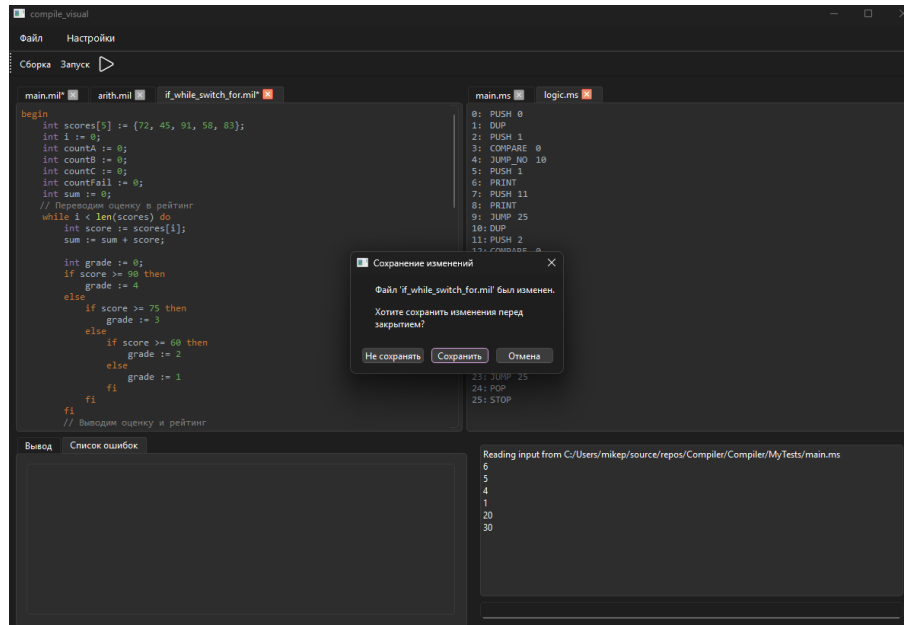


Рис. 12. Вывод диалога при закрытии программы с несохранёнными файлами

3.2 Стандартные операторы

На листинге 17 отображен код программы содержащей базовые операторы языка MiLan, а на рисунке 13 представлен вывод этой программы.

Листинг 17. Стандартные операторы

```

1 Begin
2   int i := 0;
3   while i < 10 do
4     i := i + 1;
5     if i > 5 then
6       write(i)
7     else
8       write(-i)
9     fi
10  od
11
12
13
14 End

```



Рис. 13. Результат компиляции и запуска программы 1

3.3 Цикл for

На листинге 18 отображен код программы содержащей цикл for, а на рисунке 14 представлен вывод этой программы.

Листинг 18. Цикл for

```
1 Begin
2   for (int i := 0; i < 40; i := i + 3) do
3     if i - (i / 2) * 2 = 1 then
4       write (i)
5     fi
6   od
7 End
```

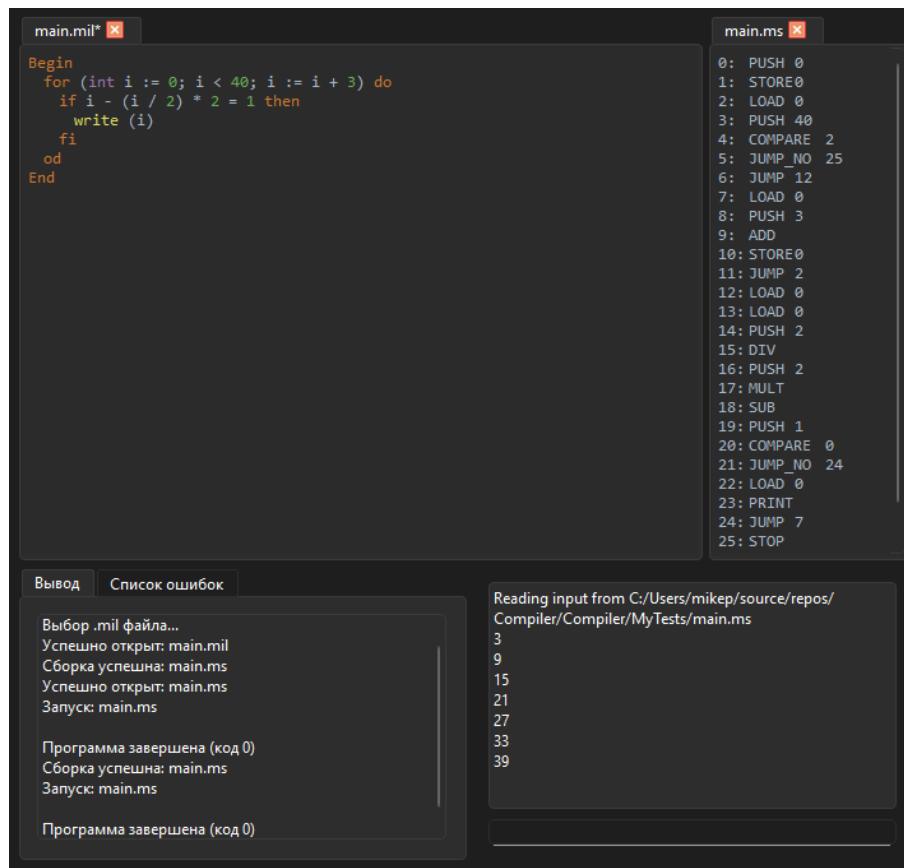


Рис. 14. Результат компиляции и запуска программы 2

3.4 Булевы константы и операции над ними

На листинге 19 представлен код программы содержащей булевы константы и операции над ними, а на рисунке 15 представлен вывод этой программы.

Листинг 19. Булевы константы и операции

```

1 begin
2   int x := 5;
3   int y := 10;
4   bool a := x < y;
5   bool b := x = y;
6   bool c := x != y && y > 0;
7   bool d := false || x > 3;
8   write(a);
9   write(b);
10  write(c);
11  write(d)
12 end

```

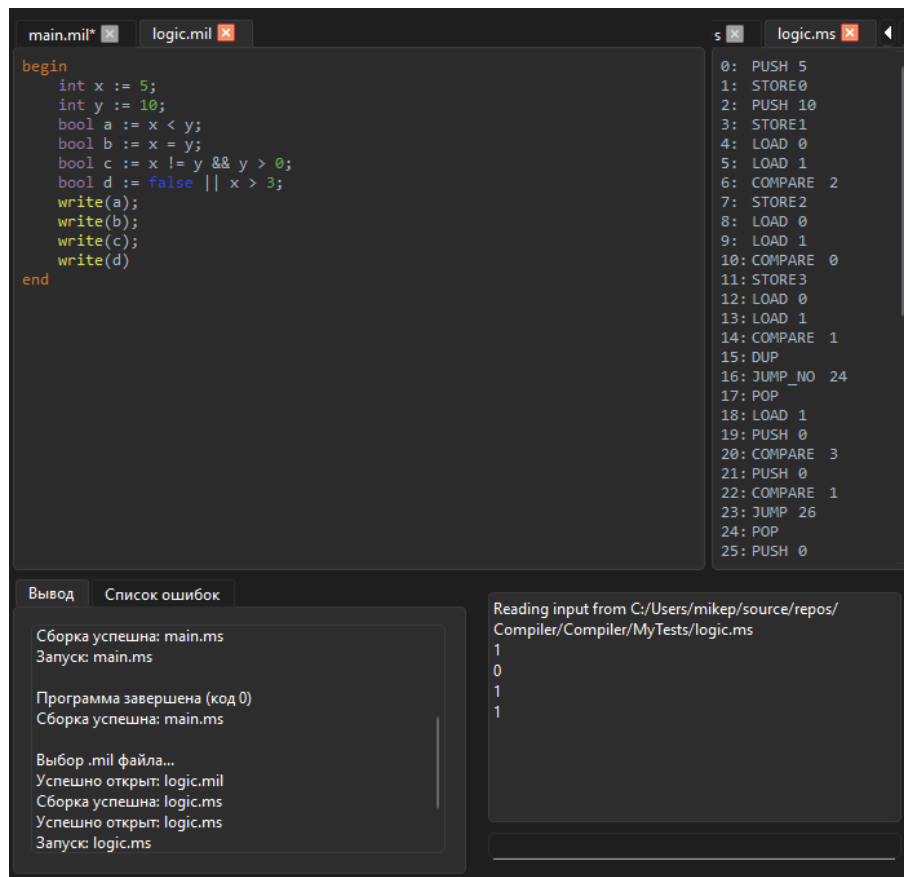


Рис. 15. Результат компиляции и запуска программы 3

3.5 Одномерные массивы и операции над ними

На листинге 20 представлен код программы содержащей одномерные массивы и операции над ними, а на рисунке 16 представлен вывод этой программы.

Листинг 20. Одномерные массивы

```
1 begin
2   int arr[5] := {10, 20, 30, 40, 50};
3   write(arr[0:2]);
4   fill(arr, 0);
5   write(arr[0:2]);
6   replace(arr, 0, 999);
7   write(arr[0]);
8   write(111111);
9   write(resize(resize(arr, 2, 0), 5, -1))
10
11 end
```

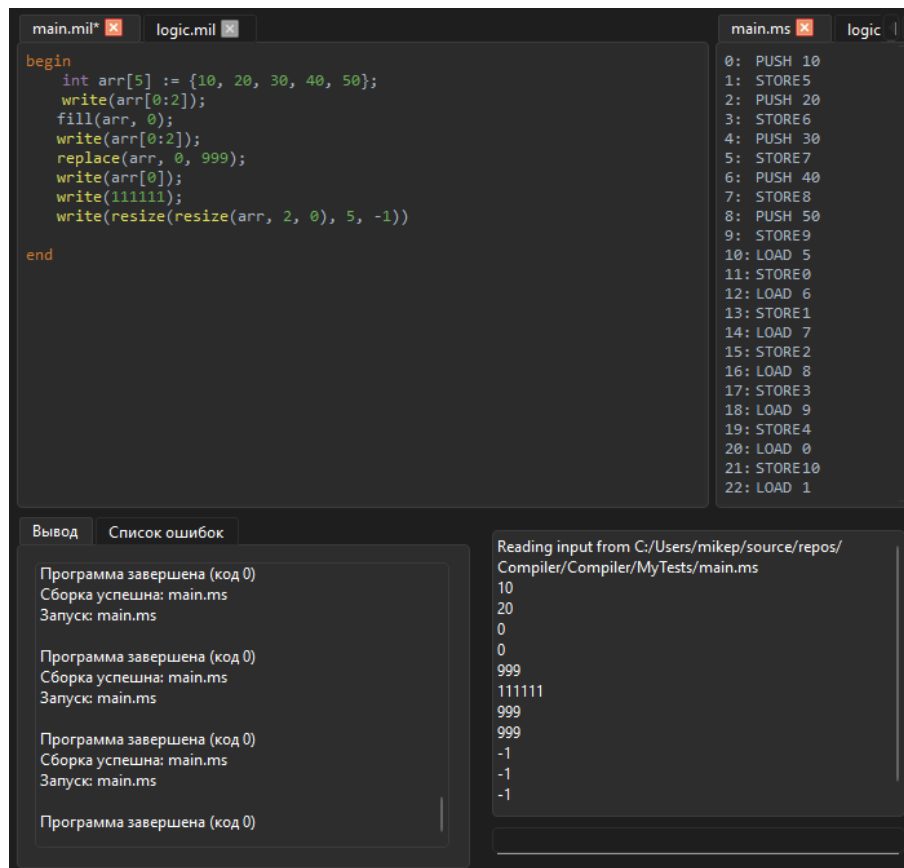


Рис. 16. Результаты компиляции и запуска программы 4

3.6 Двумерные массивы и операции над ними

На листинге 19 можно увидеть код программы содержащей двумерные массивы и операции над ними, а также умножение по правилам Шимбелла и комментарии.

На рисунке 17 представлен вывод этой программы.

Листинг 21. Двумерные массивы, комментарии, Шимбелл

```

1 begin
2   int a[2][2] := {{1, 0}, {2, 3}};
3   int b[2][2] := {{7, 4}, {4, 1}};
4
5   int c[2][2];
6   c := a * b;
7   write(c);
8   // Умножение по правилам Шимбелла
9   write(shimb(a, c, 1));
10  write(tovector(a))
11  /* многострочный
12  комментарий
13  */
14 end

```

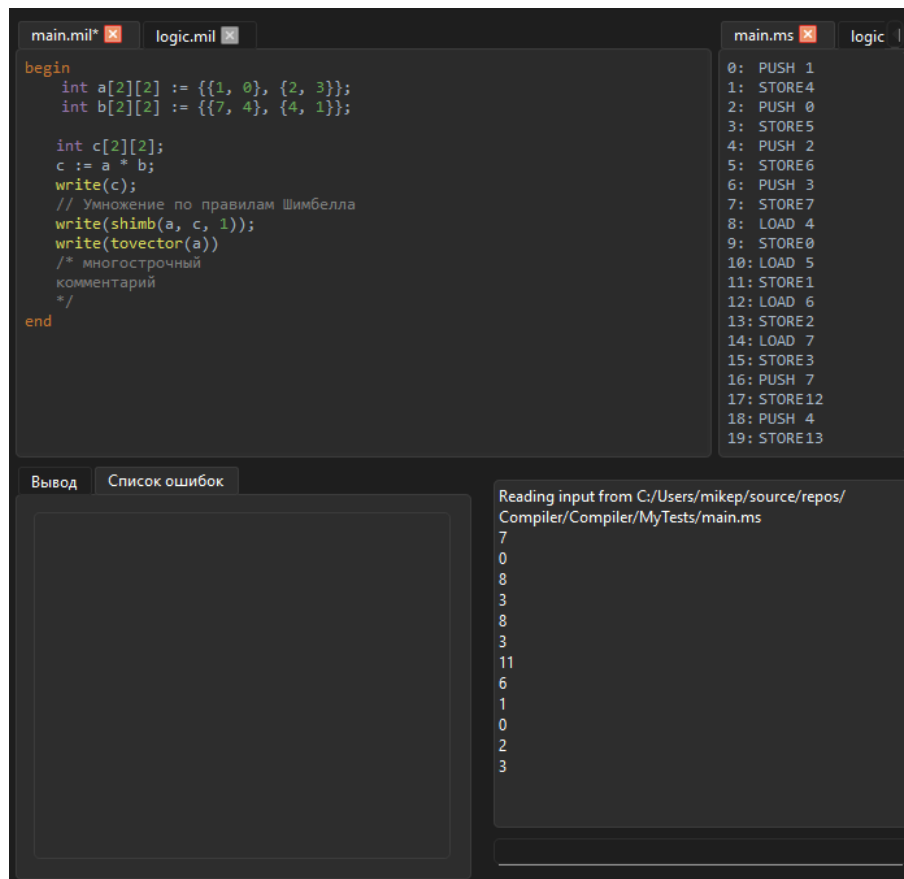


Рис. 17. Результаты компиляции и запуска программы 5

3.7 Конструкция switch ... case

На листинге 20 приведён код программы содержащей конструкцию sw, а также умножение по правилам Шимбелла и комментарии.

На рисунке 18 представлен вывод этой программы.

Листинг 22. Конструкция switch ... case

```

1  begin
2  int i := 2;
3  switch (i) begin
4      case 1: write(1) break;
5      case 2: write(2); write(2) break;
6      case 3: write(3) break;
7      case 4: write(4) break;
8      default: write(-1)
9  endcase
10 int i := 90;
11 switch (i) begin
12     case 1: write(1) break;
13     case 2: write(2) break;
14     case 3: write(3) break;
15     case 4: write(4) break;
16     default: write(-1)

```

```
17     endcase
18 end
```

The screenshot shows a compiler IDE with three main panels. The top-left panel displays the source code for `main.mil` and `logic.mil`. The top-right panel shows the assembly code for `main.ms`. The bottom panel is divided into two sections: 'Вывод' (Output) and 'Список ошибок' (Error List). The 'Вывод' section shows the output of the program, which is '2', '2', and '-1'. The 'Список ошибок' section is empty.

```
main.mil* x logic.mil x
begin
  int i := 2;
  switch (i) begin
    case 1: write(1) break;
    case 2: write(2); write(2) break;
    case 3: write(3) break;
    case 4: write(4) break;
    default: write(-1)
  endcase
  int i := 90;
  switch (i) begin
    case 1: write(1) break;
    case 2: write(2) break;
    case 3: write(3) break;
    case 4: write(4) break;
    default: write(-1)
  endcase
end

main.ms x logic |
0: PUSH 2
1: STORE 0
2: LOAD 0
3: DUP
4: PUSH 1
5: COMPARE 0
6: JUMP_NO 10
7: PUSH 1
8: PRINT
9: JUMP 37
10: DUP
11: PUSH 2
12: COMPARE 0
13: JUMP_NO 19
14: PUSH 2
15: PRINT
16: PUSH 2
17: PRINT
18: JUMP 37
19: DUP

Вывод Список ошибок
Reading input from C:/Users/mikep/source/repos/Compiler/Compiler/MyTests/main.ms
2
2
-1
```

Рис. 18. Результаты компиляции и запуска программы 6

Заключение

В ходе выполнения данной лабораторной работы были проанализированы существующие языки программирования C++, Ruby и JavaScript, а также изучена реализация операций, которыми необходимо было расширить язык программирования MiLan. Были реализованы следующие типы данных, конструкции, операторы:

- Оператор выбора.
- Оператор цикла `for`.
- Булевы константы, переменные и операции на ними.
- Одномерные и двумерные массивы и операции над ними.
- Перемножение матриц по правилам Шимбелла и все виды комментариев.
- Арифметические поэлементные операции над одномерными и двумерными массивами.

Все операции над одномерными и двумерными массивами были реализованы как «опасные» методы из ЯП Ruby, что позволяет данным операторам и менять исходный объект массива и также возвращать изменённый объект (то есть операторы не являются ни мутирующими ни иммутабельными в классическом понимании). Данная реализация позволяет уменьшить размер исходного кода программы, но требует от программиста большей внимательности при написании кода.

Расширенная грамматика ЯП MiLan была записана в БНФ-нотации, а также для её правил были составлены синтаксические диаграммы.

Лексический анализатор для распознавания лексем в программе был реализован с помощью автомата, что обеспечивает высокую скорость лексического анализа и возможность использовать динамическую подсветку лексем в коде программы.

Грамматика языка MiLan относится ко второму типу в иерархии Хомского (контекстно-свободная грамматика). Для синтаксического анализа был использован метод рекурсивного спуска. Данный метод реализует работу анализатора LL(1).

Достоинства реализации:

- Использование конечного автомата при лексическом анализе обеспечивает высокую скорость определения лексем в тексте программы, что позволяет организовать автоматическую подсветку лексем, за минимальное время.
- Реализованная среда разработки позволяет более удобно редактировать код, так как в ней реализована подсветка лексем.

Недостатки реализации:

- Одномерные и двумерные массивы реализованы только со статической памятью, что сильно ограничивает их использование в программах.

Масштабирование:

- Добавить возможность создания переменных в динамической памяти и работы с указателями.
- Реализовать другие типы данных, например `float` и `string`.
- Добавить этап оптимизации.

Список использованной литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 29.04.2026).

Приложение А. Исходный код метода initTM

```
1 inline FSM<std::wstring>* initTm(
2     std::wstring& buffer,
3     Group& currentGroup,
4     std::vector<Lexem*>& lexemTable,
5     std::vector<Error*>& errorTable,
6     std::map<int, int>& lexemsR
7 ) {
8     FSM<std::wstring>::TransitionMap tM;
9     FSM<std::wstring>* fsm = new FSM<std::wstring>();
10
11
12     auto addToLexemsR = [&lexemsR]() {
13         lexemsR.insert({ currentRow, currentLexem });
14         currentLexem++;
15     };
16
17
18     auto y1 = [&fsm](wchar_t c) {fsm->popFront();};
19     auto y2 = [&buffer](wchar_t c) { buffer += c; };
20     auto y3 = [&buffer](wchar_t c) { buffer.clear(); };
21     auto y4 = [&buffer](wchar_t c) { currentRow++; currentPos = 1; };
22     auto y_Div = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
23         lexemTable.push_back(&lDiv); addToLexemsR(); };
24     auto y_Id = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
25         lexemTable.push_back(new Lexem(buffer, Id)); addToLexemsR(); };
26     auto y_Plus = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
27         lexemTable.push_back(&lPlus); addToLexemsR(); };
28     auto y_Diff = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
29         lexemTable.push_back(&lDiff); addToLexemsR(); };
30     auto y_Mult = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
31         lexemTable.push_back(&lMult); addToLexemsR(); };
32     auto y_LBracket = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
33         lexemTable.push_back(&lLBracket); addToLexemsR(); };
34     auto y_RBracket = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
35         lexemTable.push_back(&lRBracket); addToLexemsR(); };
36     auto y_LCurly = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
37         lexemTable.push_back(&lLCurly); addToLexemsR(); };
38     auto y_RCurly = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
39         lexemTable.push_back(&RCurly); addToLexemsR(); };
40     auto y_LSquare = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
41         lexemTable.push_back(&LSquare); addToLexemsR(); };
42     auto y_RSquare = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
43         lexemTable.push_back(&RSquare); addToLexemsR(); };
44     auto y_Assign = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
45         lexemTable.push_back(&lAssign); addToLexemsR(); };
46     auto y_Semicolon = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
47         lexemTable.push_back(&lSemicolon); addToLexemsR(); };
48     auto y_Comma = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
49         lexemTable.push_back(&lComma); addToLexemsR(); };
50     auto y_Var = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
51         lexemTable.push_back(new Lexem(buffer, Var)); addToLexemsR(); };
52     auto y_ShortAnd = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
53         lexemTable.push_back(&lShortAnd); addToLexemsR(); };
54     auto y_FullAnd = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
```

```

    lexemTable.push_back(&lFullAnd); addToLexemsR(); };
39 auto y_ShortOr = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lShortOr); addToLexemsR(); };
40 auto y_FullOr = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lFullOr); addToLexemsR(); };
41 auto y_Equal = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lEqual); addToLexemsR(); };
42 auto y_NotEqual = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lNotEqual); addToLexemsR(); };
43 auto y_Not = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lNot); addToLexemsR(); };
44 auto y_LessEqual = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lLessEqual); addToLexemsR(); };
45 auto y_Less = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lLess); addToLexemsR(); };
46 auto y_GreaterEqual = [&lexemTable, &buffer, &addToLexemsR](wchar_t c
    ) { lexemTable.push_back(&lGreaterEqual); addToLexemsR(); };
47 auto y_Greater = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lGreater); addToLexemsR(); };
48 auto y_Begin = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lBegin); addToLexemsR(); };
49 auto y_Break = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lBreak); addToLexemsR(); };
50 auto y_Bool = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lBool); addToLexemsR(); };
51 auto y_Case = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lCase); addToLexemsR(); };
52 auto y_Continue = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lContinue); addToLexemsR(); };
53 auto y_Do = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lDo); addToLexemsR(); };
54 auto y_End = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lEnd); addToLexemsR(); };
55 auto y_Else = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lElse); addToLexemsR(); };
56 auto y_For = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lFor); addToLexemsR(); };
57 auto y_False = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lFalse); addToLexemsR(); };
58 auto y_Find = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lFind); addToLexemsR(); };
59 auto y_Fi = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lFi); addToLexemsR(); };
60 auto y_Int = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lInt); addToLexemsR(); };
61 auto y_If = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lIf); addToLexemsR(); };
62 auto y_Len = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lLen); addToLexemsR(); };
63 auto y_0d = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&l0d); addToLexemsR(); };
64 auto y_Replace = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lReplace); addToLexemsR(); };
65 auto y_Reverse = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
    lexemTable.push_back(&lReverse); addToLexemsR(); };
66 auto y_Resize = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {

```

```

        lexemTable.push_back(&lResize); addToLexemsR(); };
67 auto y_Read = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lRead); addToLexemsR(); };
68 auto y_Switch = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lSwitch); addToLexemsR(); };
69 auto y_Shimb = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lShimb); addToLexemsR(); };
70 auto y_True = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lTrue); addToLexemsR(); };
71 auto y_Then = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lThen); addToLexemsR(); };
72 auto y_While = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lWhile); addToLexemsR(); };
73 auto y_Write = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lWrite); addToLexemsR(); };
74 auto y_Trans = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lTrans); addToLexemsR(); };
75
76 auto y_Colon = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lColon); addToLexemsR(); };
77 auto y_ToVector = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lToVector); addToLexemsR(); };
78 auto y_Default = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lDefault); addToLexemsR(); };
79 auto y_Sorted = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lSorted); addToLexemsR(); };
80 auto y_Comment = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(new Lexem(buffer, Comment)); };
81 auto y_MultiComment = [&lexemTable, &buffer, &addToLexemsR](wchar_t c
        ) { lexemTable.push_back(new Lexem(buffer, MultiComment)); };
82
83 auto y_Fill = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lFill); addToLexemsR(); };
84 auto y_Copy = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lCopy); addToLexemsR(); };
85 auto y_EndCase = [&lexemTable, &buffer, &addToLexemsR](wchar_t c) {
        lexemTable.push_back(&lEndCase); addToLexemsR(); };
86
87 auto y_GroupNone = [&currentGroup](wchar_t c) { currentGroup = gNone;
        };
88 auto y_GroupComment = [&currentGroup](wchar_t c) { currentGroup =
        gComment; };
89 auto y_GroupMultiComment = [&currentGroup](wchar_t c) { currentGroup
        = gMultiComment; };
90 auto y_GroupId = [&currentGroup](wchar_t c) { currentGroup = gId; };
91 auto y_GroupVar = [&currentGroup](wchar_t c) { currentGroup = gVar; }
        ;
92 auto y_GroupWord = [&currentGroup](wchar_t c) { currentGroup = gWord;
        };
93
94
95 tM[0] = {
96     {1, {is_ws, {y1}}},
97     {2, {[](wchar_t c) {return c == L'/';} , {y2, y1}}},
98     {15, {[](wchar_t c) {return c == L'+';} , {y_Plus, y1}}},
99     {16, {[](wchar_t c) {return c == L'-';} , {y_Diff, y1}}},

```

```

100 {17, {[[] (wchar_t c) {return c == L '*' ; } , {y_Mult, y1}}} ,
101 {18, {[[] (wchar_t c) {return c == L '(' ; } , {y_LBracket, y1}}} ,
102 {19, {[[] (wchar_t c) {return c == L ')' ; } , {y_RBracket, y1}}} ,
103 {20, {[[] (wchar_t c) {return c == L '{' ; } , {y_LCurly, y1}}} ,
104 {21, {[[] (wchar_t c) {return c == L '}' ; } , {y_RCurly, y1}}} ,
105 {22, {[[] (wchar_t c) {return c == L '[' ; } , {y_LSquare, y1}}} ,
106 {23, {[[] (wchar_t c) {return c == L ']' ; } , {y_RSquare, y1}}} ,
107 {24, {[[] (wchar_t c) {return c == L ':' ; } , {y2, y1}}} ,
108 {26, {[[] (wchar_t c) {return c == L ';' ; } , {y_Semicolon, y1}}} ,
109 {27, {[[] (wchar_t c) {return c == L ',' ; } , {y_Comma, y1}}} ,
110 {28, {is_digit, {y_GroupVar, y2, y1}}} ,
111 {36, {[[] (wchar_t c) {return c == L '&' ; } , {y2, y1}}} ,
112 {39, {[[] (wchar_t c) {return c == L '|' ; } , {y2, y1}}} ,
113 {42, {[[] (wchar_t c) {return c == L '=' ; } , {y_Equal, y1}}} ,
114 {43, {[[] (wchar_t c) {return c == L '!' ; } , {y2, y1}}} ,
115 {45, {[[] (wchar_t c) {return c == L '<' ; } , {y2, y1}}} ,
116 {48, {[[] (wchar_t c) {return c == L '>' ; } , {y2, y1}}} ,
117 {51, {[[] (wchar_t c) {return t1(c) == L 'b' ; } , {y_GroupWord, y2,
118 y1}}} ,
118 {60, {[[] (wchar_t c) {return t1(c) == L 'c' ; } , {y_GroupWord, y2,
119 y1}}} ,
119 {71, {[[] (wchar_t c) {return t1(c) == L 'd' ; } , {y_GroupWord, y2,
120 y1}}} ,
120 {73, {[[] (wchar_t c) {return t1(c) == L 'e' ; } , {y_GroupWord, y2,
121 y1}}} ,
121 {79, {[[] (wchar_t c) {return t1(c) == L 'f' ; } , {y_GroupWord, y2,
122 y1}}} ,
122 {94, {[[] (wchar_t c) {return t1(c) == L 'i' ; } , {y_GroupWord, y2,
123 y1}}} ,
123 {98, {[[] (wchar_t c) {return t1(c) == L 'l' ; } , {y_GroupWord, y2,
124 y1}}} ,
124 {101, {[[] (wchar_t c) {return t1(c) == L 'o' ; } , {y_GroupWord, y2,
125 y1}}} ,
125 {103, {[[] (wchar_t c) {return t1(c) == L 'r' ; } , {y_GroupWord, y2,
126 y1}}} ,
126 {125, {[[] (wchar_t c) {return t1(c) == L 's' ; } , {y_GroupWord, y2,
127 y1}}} ,
127 {140, {[[] (wchar_t c) {return t1(c) == L 't' ; } , {y_GroupWord, y2,
128 y1}}} ,
128 {147, {[[] (wchar_t c) {return t1(c) == L 'w' ; } , {y_GroupWord, y2,
129 y1}}} ,
129 {12, {[[] (wchar_t c) {return is_letter(t1(c)); } , {y_GroupId, y2,
130 y1}}} ,
130 } ;
131
132 tM[1] = {
133 {0, {ret_true, {y_GroupNone}}}
134 } ;
135 tM[2] = {
136 {3, {[[] (wchar_t c) {return c == L '/' ; } , {y_GroupComment, y2, y1}
137 }} ,
137 {6, {[[] (wchar_t c) {return c != L '/' && c != L '*' ; } , {y_Div, y3}
138 }} ,
138 {7, {[[] (wchar_t c) {return c == L '*' ; } , {y_GroupMultiComment, y2
139 , y1}}}
139 } ;

```

```

140     tM[3] = {
141         {4, {[](wchar_t c) {return c != L'\n'; } , {y2, y1}}},
142         {5, {[](wchar_t c) {return c == L'\n'; } , {y4, y_Comment, y3, y1}
143         }};
144     tM[4] = {
145         {4, {[](wchar_t c) {return c != L'\n'; } , {y2, y1}}},
146         {5, {[](wchar_t c) {return c == L'\n'; } , {y4, y_Comment, y3, y1}
147         }};
148     tM[5] = {
149         {0, {ret_true, {y_GroupNone}}}
150     };
151     tM[6] = {
152         {0, {ret_true, {y_GroupNone}}}
153     };
154     tM[7] = {
155         {8, {[](wchar_t c) {return c == L'\n'; } , {y4, y2, y1}}},
156         {8, {[](wchar_t c) {return c != L'*'; } , {y2, y1}}},
157         {9, {[](wchar_t c) {return c == L'*'; } , {y2, y1}}}
158     };
159     tM[8] = {
160         {8, {[](wchar_t c) {return c == L'\n'; } , {y4, y2, y1}}},
161         {8, {[](wchar_t c) {return c != L'*'; } , {y2, y1}}},
162         {9, {[](wchar_t c) {return c == L'*'; } , {y2, y1}}}
163     };
164     tM[9] = {
165         {10, {[](wchar_t c) {return c == L'/' ; } , {y2, y_MultiComment, y3
166         , y1}}},
167         {11, {[](wchar_t c) {return c == L'\n'; } , {y4, y2, y1}}},
168         {11, {[](wchar_t c) {return is_ws(c) or c != L'/' ; } , {y2, y1}}}
169     };
170     tM[10] = {
171         {0, {ret_true, {y_GroupNone}}}
172     };
173     tM[11] = {
174         {8, {[](wchar_t c) {return c == L'\n'; } , {y4, y2, y1}}},
175         {8, {[](wchar_t c) {return c != L'*'; } , {y2, y1}}},
176         {9, {[](wchar_t c) {return c == L'*'; } , {y2, y1}}}
177     };
178     tM[12] = {
179         {13, {[](wchar_t c) {return not (is_digit(c) or is_letter(tl(c)));
180         } , {y_Id, y3}}},
181         {14, {[](wchar_t c) {return is_digit(c) or is_letter(tl(c)); } , {
182         y2, y1}}}
183     };
184     tM[13] = {
185         {0, {ret_true, {y_GroupNone}}}
186     };
187     tM[14] = {
188         {13, {[](wchar_t c) {return not (is_digit(c) or is_letter(tl(c)));
189         } , {y_Id, y3}}},
190         {14, {[](wchar_t c) {return is_digit(c) or is_letter(tl(c)); } , {
191         y2, y1}}}
192     };
193     tM[15] = {

```

```

189     {0, {ret_true, {y_GroupNone}}}}
190 };
191 tM[16] = {
192     {0, {ret_true, {y_GroupNone}}}}
193 };
194 tM[17] = {
195     {0, {ret_true, {y_GroupNone}}}}
196 };
197 tM[18] = {
198     {0, {ret_true, {y_GroupNone}}}}
199 };
200 tM[19] = {
201     {0, {ret_true, {y_GroupNone}}}}
202 };
203 tM[20] = {
204     {0, {ret_true, {y_GroupNone}}}}
205 };
206 tM[21] = {
207     {0, {ret_true, {y_GroupNone}}}}
208 };
209 tM[22] = {
210     {0, {ret_true, {y_GroupNone}}}}
211 };
212 tM[23] = {
213     {0, {ret_true, {y_GroupNone}}}}
214 };
215 tM[24] = {
216     {25, {[](wchar_t c) {return c == L'='; } , {y_Assign, y3, y1}}},
217     {157, {[](wchar_t c) {return c != L'='; } , {y_Colon, y3}}}}
218 };
219 tM[25] = {
220     {0, {ret_true, {y_GroupNone}}}}
221 };
222 tM[26] = {
223     {0, {ret_true, {y_GroupNone}}}}
224 };
225 tM[27] = {
226     {0, {ret_true, {y_GroupNone}}}}
227 };
228 tM[28] = {
229     {28, {is_digit, {y2, y1}}},
230     {29, {[](wchar_t c) {return not is_digit(c) and not is_letter(tl(c
231         )); } , {y_Var, y3}}}}
232 };
233 tM[29] = {
234     {0, {ret_true, {y_GroupNone}}}}
235 };
236 tM[36] = {
237     {37, {[](wchar_t c) {return c == L'&'; } , {y_ShortAnd, y3, y1}}},
238     {38, {[](wchar_t c) {return c != L'&'; } , {y_FullAnd, y3}}}}
239 };
240 tM[37] = {
241     {0, {ret_true, {y_GroupNone}}}}
242 };
243 tM[38] = {
244     {0, {ret_true, {y_GroupNone}}}}

```

```

244     };
245     tM[39] = {
246         {40, {[](wchar_t c) {return c == L'|'; } , {y_ShortOr, y3, y1}}} ,
247         {41, {[](wchar_t c) {return c != L'|'; } , {y_FullOr, y3}}}
248     };
249     tM[40] = {
250         {0, {ret_true, {y_GroupNone}}}
251     };
252     tM[41] = {
253         {0, {ret_true, {y_GroupNone}}}
254     };
255     tM[42] = {
256         {0, {ret_true, {y_GroupNone}}}
257     };
258     tM[43] = {
259         {44, {[](wchar_t c) {return c == L'='; } , {y_NotEqual, y3, y1}}} ,
260         {156, {[](wchar_t c) {return c != L'='; } , {y_Not, y3}}}
261     };
262     tM[44] = {
263         {0, {ret_true, {y_GroupNone}}}
264     };
265     tM[156] = {
266         {0, {ret_true, {y_GroupNone}}}
267     };
268     tM[45] = {
269         {46, {[](wchar_t c) {return c == L'='; } , {y_LessEqual, y3, y1}}} ,
270         {47, {[](wchar_t c) {return c != L'='; } , {y_Less, y3}}}
271     };
272     tM[46] = {
273         {0, {ret_true, {y_GroupNone}}}
274     };
275     tM[47] = {
276         {0, {ret_true, {y_GroupNone}}}
277     };
278     tM[48] = {
279         {49, {[](wchar_t c) {return c == L'='; } , {y_GreaterEqual, y3, y1}}}
280         ,
281         {50, {[](wchar_t c) {return c != L'='; } , {y_Greater, y3}}}
282     };
283     tM[49] = {
284         {0, {ret_true, {y_GroupNone}}}
285     };
286     tM[50] = {
287         {0, {ret_true, {y_GroupNone}}}
288     };
289     tM[51] = {
290         {52, {[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}}} ,
291         {56, {[](wchar_t c) {return t1(c) == L'r'; } , {y2, y1}}} ,
292         {82, {[](wchar_t c) {return t1(c) == L'o'; } , {y2, y1}}}
293     };
294     tM[52] = {
295         {53, {[](wchar_t c) {return t1(c) == L'g'; } , {y2, y1}}}
296     };
297     tM[53] = {
298         {54, {[](wchar_t c) {return t1(c) == L'i'; } , {y2, y1}}}
299     };

```

```

299     tM[54] = {
300         {55, {[[](wchar_t c) {return t1(c) == L'n'; } , {y_Begin, y3, y1}} }
301     };
302     tM[55] = {
303         {0, {ret_true, {} } }
304     };
305     tM[56] = {
306         {57, {[[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}} }
307     };
308     tM[57] = {
309         {58, {[[](wchar_t c) {return t1(c) == L'a'; } , {y2, y1}} }
310     };
311     tM[58] = {
312         {59, {[[](wchar_t c) {return t1(c) == L'k'; } , {y_Break, y3, y1}} }
313     };
314     tM[59] = {
315         {0, {ret_true, {} } }
316     };
317     tM[60] = {
318         {61, {[[](wchar_t c) {return t1(c) == L'a'; } , {y2, y1}} } ,
319         {64, {[[](wchar_t c) {return t1(c) == L'o'; } , {y2, y1}} }
320     };
321     tM[61] = {
322         {62, {[[](wchar_t c) {return t1(c) == L's'; } , {y2, y1}} }
323     };
324     tM[62] = {
325         {63, {[[](wchar_t c) {return t1(c) == L'e'; } , {y_Case, y3, y1}} }
326     };
327     tM[63] = {
328         {0, {ret_true, {} } }
329     };
330     tM[64] = {
331         {65, {[[](wchar_t c) {return t1(c) == L'n'; } , {y2, y1}} } ,
332         {32, {[[](wchar_t c) {return t1(c) == L'p'; } , {y2, y1}} }
333     };
334     tM[32] = {
335         {33, {[[](wchar_t c) {return t1(c) == L'y'; } , {y_Copy, y3, y1}}}
336     };
337     tM[33] = {
338         {0, {ret_true, {} } }
339     };
340     tM[65] = {
341         {66, {[[](wchar_t c) {return t1(c) == L't'; } , {y2, y1}} }
342     };
343     tM[66] = {
344         {67, {[[](wchar_t c) {return t1(c) == L'i'; } , {y2, y1}} }
345     };
346     tM[67] = {
347         {68, {[[](wchar_t c) {return t1(c) == L'n'; } , {y2, y1}} }
348     };
349     tM[68] = {
350         {69, {[[](wchar_t c) {return t1(c) == L'u'; } , {y2, y1}} }
351     };
352     tM[69] = {
353         {70, {[[](wchar_t c) {return t1(c) == L'e'; } , {y_Continue, y3, y1}
354             } }

```

```

354     };
355     tM[70] = {
356         {0, {ret_true, {} } }
357     };
358     tM[71] = {
359         {72, {[](wchar_t c) {return t1(c) == L'o'; } , {y_Do, y3, y1}} } ,
360         {158, {[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}}}
361     };
362     tM[72] = {
363         {0, {ret_true, {} } }
364     };
365     tM[73] = {
366         {74, {[](wchar_t c) {return t1(c) == L'n'; } , {y2, y1}} } ,
367         {76, {[](wchar_t c) {return t1(c) == L'l'; } , {y2, y1}} }
368     };
369     tM[74] = {
370         {75, {[](wchar_t c) {return t1(c) == L'd'; } , {y1}} }
371     };
372     tM[75] = {
373         {179, {[](wchar_t c) {return t1(c) == L'c'; } , {y2, y1}} } ,
374         {0, {ret_true, {y_End, y3}} }
375     };
376     tM[179] = {
377         {180, {[](wchar_t c) {return t1(c) == L'a'; } , {y2, y1}} } ,
378     };
379     tM[180] = {
380         {181, {[](wchar_t c) {return t1(c) == L's'; } , {y2, y1}} } ,
381     };
382     tM[181] = {
383         {182, {[](wchar_t c) {return t1(c) == L'e'; } , {y_EndCase, y3, y1}}
384         } } ,
385     };
386     tM[182] = {
387         {0, {ret_true, {} } }
388     };
389     tM[76] = {
390         {77, {[](wchar_t c) {return t1(c) == L's'; } , {y2, y1}} }
391     };
392     tM[77] = {
393         {78, {[](wchar_t c) {return t1(c) == L'e'; } , {y_Else, y3, y1}} }
394     };
395     tM[78] = {
396         {0, {ret_true, {} } }
397     };
398     tM[79] = {
399         {80, {[](wchar_t c) {return t1(c) == L'o'; } , {y2, y1}} } ,
400         {86, {[](wchar_t c) {return t1(c) == L'a'; } , {y2, y1}} } ,
401         {90, {[](wchar_t c) {return t1(c) == L'i'; } , {y2, y1}} }
402     };
403     tM[80] = {
404         {81, {[](wchar_t c) {return t1(c) == L'r'; } , {y_For, y3, y1}} }
405     };
406     tM[81] = {
407         {0, {ret_true, {} } }
408     };
409     tM[86] = {

```

```

409     {87, {[[](wchar_t c) {return t1(c) == L'l'; } , {y2, y1}} }
410 };
411     tM[87] = {
412         {88, {[[](wchar_t c) {return t1(c) == L's'; } , {y2, y1}} }
413     };
414     tM[88] = {
415         {89, {[[](wchar_t c) {return t1(c) == L'e'; } , {y_False, y3, y1}} }
416     };
417     tM[89] = {
418         {0, {ret_true, {} }
419     };
420     tM[90] = {
421         {91, {[[](wchar_t c) {return t1(c) == L'n'; } , {y2, y1}} } ,
422         {93, {[[](wchar_t c) {return not(is_digit(c) or is_letter(t1(c)));
423             } , {y_Fi, y3}} } ,
424         {30, {[[](wchar_t c) {return t1(c) == L'l'; } , {y2, y1}}}
425     };
426     tM[30] = {
427         {31, {[[](wchar_t c) {return t1(c) == L'l'; } , {y_Fill, y3, y1}}}
428     };
429     tM[31] = {
430         {0, {ret_true, {} }
431     };
432     tM[91] = {
433         {92, {[[](wchar_t c) {return t1(c) == L'd'; } , {y_Find, y3, y1}} }
434     };
435     tM[92] = {
436         {0, {ret_true, {} }
437     };
438     tM[93] = {
439         {0, {ret_true, {} }
440     };
441     tM[94] = {
442         {95, {[[](wchar_t c) {return t1(c) == L'n'; } , {y2, y1}} } ,
443         {97, {[[](wchar_t c) {return t1(c) == L'f'; } , {y_If, y3, y1}} }
444     };
445     tM[95] = {
446         {96, {[[](wchar_t c) {return t1(c) == L't'; } , {y_Int, y3, y1}} }
447     };
448     tM[96] = {
449         {0, {ret_true, {} }
450     };
451     tM[97] = {
452         {0, {ret_true, {} }
453     };
454     tM[98] = {
455         {99, {[[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}} } ,
456     };
457     tM[99] = {
458         {100, {[[](wchar_t c) {return t1(c) == L'n'; } , {y_Len, y3, y1}} }
459     };
460     tM[100] = {
461         {0, {ret_true, {} }
462     };
463     tM[101] = {
464         {102, {[[](wchar_t c) {return t1(c) == L'd'; } , {y_0d, y3, y1}} } ,

```

```

464     };
465     tM[102] = {
466         {0, {ret_true, {} } }
467     };
468     tM[103] = {
469         {104, {[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}} } ,
470     };
471     tM[104] = {
472         {105, {[](wchar_t c) {return t1(c) == L'p'; } , {y2, y1}} } ,
473         {110, {[](wchar_t c) {return t1(c) == L'v'; } , {y2, y1}} } ,
474         {115, {[](wchar_t c) {return t1(c) == L's'; } , {y2, y1}} } ,
475         {119, {[](wchar_t c) {return t1(c) == L'a'; } , {y2, y1}} }
476     };
477     tM[105] = {
478         {106, {[](wchar_t c) {return t1(c) == L'l'; } , {y2, y1}} }
479     };
480     tM[106] = {
481         {107, {[](wchar_t c) {return t1(c) == L'a'; } , {y2, y1}} }
482     };
483     tM[107] = {
484         {108, {[](wchar_t c) {return t1(c) == L'c'; } , {y2, y1}} }
485     };
486     tM[108] = {
487         {109, {[](wchar_t c) {return t1(c) == L'e'; } , {y_Replace, y3, y1}
488             } }
489     };
490     tM[109] = {
491         {0, {ret_true, {} } }
492     };
493     tM[110] = {
494         {111, {[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}} }
495     };
496     tM[111] = {
497         {112, {[](wchar_t c) {return t1(c) == L'r'; } , {y2, y1}} }
498     };
499     tM[112] = {
500         {113, {[](wchar_t c) {return t1(c) == L's'; } , {y2, y1}} }
501     };
502     tM[113] = {
503         {114, {[](wchar_t c) {return t1(c) == L'e'; } , {y_Reverse, y3, y1}
504             } }
505     };
506     tM[114] = {
507         {0, {ret_true, {} } }
508     };
509     tM[115] = {
510         {116, {[](wchar_t c) {return t1(c) == L'i'; } , {y2, y1}} }
511     };
512     tM[116] = {
513         {117, {[](wchar_t c) {return t1(c) == L'z'; } , {y2, y1}} }
514     };
515     tM[117] = {
516         {118, {[](wchar_t c) {return t1(c) == L'e'; } , {y_Resize, y3, y1}}
517             }
518     };
519     tM[118] = {

```

```

517     {0, {ret_true, {} } }
518 };
519 tM[119] = {
520     {120, {[](wchar_t c) {return t1(c) == L'd'; } , {y_Read, y3, y1}} }
521 };
522 tM[120] = {
523     {0, {ret_true, {} } }
524 };
525 tM[125] = {
526     {131, {[](wchar_t c) {return t1(c) == L'w'; } , {y2, y1}} } ,
527     {136, {[](wchar_t c) {return t1(c) == L'h'; } , {y2, y1}} } ,
528     {164, {[](wchar_t c) {return t1(c) == L'o'; } , {y2, y1}} }
529 };
530 tM[131] = {
531     {132, {[](wchar_t c) {return t1(c) == L'i'; } , {y2, y1}} }
532 };
533 tM[132] = {
534     {133, {[](wchar_t c) {return t1(c) == L't'; } , {y2, y1}} }
535 };
536 tM[133] = {
537     {134, {[](wchar_t c) {return t1(c) == L'c'; } , {y2, y1}} }
538 };
539 tM[134] = {
540     {135, {[](wchar_t c) {return t1(c) == L'h'; } , {y_Switch, y3, y1}}
541     };
542 tM[135] = {
543     {0, {ret_true, {} } }
544 };
545 tM[136] = {
546     {137, {[](wchar_t c) {return t1(c) == L'i'; } , {y2, y1}} }
547 };
548 tM[137] = {
549     {138, {[](wchar_t c) {return t1(c) == L'm'; } , {y2, y1}} }
550 };
551 tM[138] = {
552     {139, {[](wchar_t c) {return t1(c) == L'b'; } , {y_Shimb, y3, y1}}
553     };
554 true;
555 tM[139] = {
556     {0, {ret_true, {} } }
557 };
558 tM[140] = {
559     {141, {[](wchar_t c) {return t1(c) == L'r'; } , {y2, y1}} } ,
560     {144, {[](wchar_t c) {return t1(c) == L'h'; } , {y2, y1}} } ,
561     {169, {[](wchar_t c) {return t1(c) == L'o'; } , {y2, y1}} }
562 };
563 tM[141] = {
564     {142, {[](wchar_t c) {return t1(c) == L'u'; } , {y2, y1}} } ,
565     {176, {[](wchar_t c) {return t1(c) == L'a'; } , {y2, y1}} }
566 };
567 tM[142] = {
568     {143, {[](wchar_t c) {return t1(c) == L'e'; } , {y_True, y3, y1}} }
569 };
570 tM[143] = {

```

```

571     {0, {ret_true, {} } }
572 };
573 tM[144] = {
574     {145, {[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}} }
575 };
576 tM[145] = {
577     {146, {[](wchar_t c) {return t1(c) == L'n'; } , {y_Then, y3, y1}} }
578 };
579 tM[146] = {
580     {0, {ret_true, {} } }
581 };
582 tM[147] = {
583     {148, {[](wchar_t c) {return t1(c) == L'h'; } , {y2, y1}} } ,
584     {152, {[](wchar_t c) {return t1(c) == L'r'; } , {y2, y1}} }
585 };
586 tM[148] = {
587     {149, {[](wchar_t c) {return t1(c) == L'i'; } , {y2, y1}} }
588 };
589 tM[149] = {
590     {150, {[](wchar_t c) {return t1(c) == L'l'; } , {y2, y1}} }
591 };
592 tM[150] = {
593     {151, {[](wchar_t c) {return t1(c) == L'e'; } , {y_While, y3, y1}}
594     };
595 tM[151] = {
596     {0, {ret_true, {} } }
597 };
598 tM[152] = {
599     {153, {[](wchar_t c) {return t1(c) == L'i'; } , {y2, y1}} }
600 };
601 tM[153] = {
602     {154, {[](wchar_t c) {return t1(c) == L't'; } , {y2, y1}} }
603 };
604 tM[154] = {
605     {155, {[](wchar_t c) {return t1(c) == L'e'; } , {y_Write, y3, y1}}
606     };
607 tM[155] = {
608     {0, {ret_true, {} } }
609 };
610 tM[156] = {
611     {0, {ret_true, {y_GroupNone}}}
612 };
613 tM[157] = {
614     {0, {ret_true, {y_GroupNone}}}
615 };
616 tM[158] = {
617     {159, {[](wchar_t c) {return t1(c) == L'f'; } , {y2, y1}}}
618 };
619 tM[159] = {
620     {160, {[](wchar_t c) {return t1(c) == L'a'; } , {y2, y1}}}
621 };
622 tM[160] = {
623     {161, {[](wchar_t c) {return t1(c) == L'u'; } , {y2, y1}}}
624 };

```

```

625     tM[161] = {
626         {162, {[](wchar_t c) {return t1(c) == L'1'; } , {y2, y1}}}
627     };
628     tM[162] = {
629         {163, {[](wchar_t c) {return t1(c) == L't'; } , {y_Default, y3, y1}
630         }};
631     tM[163] = {
632         {0, {ret_true, {y_GroupNone}}}}
633     };
634     tM[164] = {
635         {165, {[](wchar_t c) {return t1(c) == L'r'; } , {y2, y1}}}
636     };
637     tM[165] = {
638         {166, {[](wchar_t c) {return t1(c) == L't'; } , {y2, y1}}}
639     };
640     tM[166] = {
641         {167, {[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}}}
642     };
643     tM[167] = {
644         {168, {[](wchar_t c) {return t1(c) == L'd'; } , {y_Sorted, y3, y1}
645         }};
646     tM[168] = {
647         {0, {ret_true, {y_GroupNone}}}}
648     };
649     tM[169] = {
650         {170, {[](wchar_t c) {return t1(c) == L'v'; } , {y2, y1}}}
651     };
652     tM[170] = {
653         {171, {[](wchar_t c) {return t1(c) == L'e'; } , {y2, y1}}}
654     };
655     tM[171] = {
656         {172, {[](wchar_t c) {return t1(c) == L'c'; } , {y2, y1}}}
657     };
658     tM[172] = {
659         {173, {[](wchar_t c) {return t1(c) == L't'; } , {y2, y1}}}
660     };
661     tM[173] = {
662         {174, {[](wchar_t c) {return t1(c) == L'o'; } , {y2, y1}}}
663     };
664     tM[174] = {
665         {175, {[](wchar_t c) {return t1(c) == L'r'; } , {y_ToVector, y3, y1
666         }}};
667     tM[175] = {
668         {0, {ret_true, {y_GroupNone}}}}
669     };
670     tM[176] = {
671         {177, {[](wchar_t c) {return t1(c) == L'n'; } , {y2, y1}}}
672     };
673     tM[177] = {
674         {178, {[](wchar_t c) {return t1(c) == L'n'; } , {y_Trans, y3, y1}}}
675     };
676     tM[178] = {
677         {0, {ret_true, {} } }

```

```

678     };
679     tM[82] = {
680         {83, {[](wchar_t c) {return t1(c) == L'o'; } , {y2, y1}} }
681     };
682     tM[83] = {
683         {84, {[](wchar_t c) {return t1(c) == L'l'; } , {y_Boot, y3, y1}} }
684     };
685     tM[84] = {
686         {0, {ret_true, {}} }
687     };
688
689     auto errorProcessor = [&buffer, &currentGroup, &fsm, &errorTable, &
        lexemTable, &y_Comment, &addToLexemsR](bool isEnd) {
690         int pos_delta = 0;
691         int buffer_len = buffer.size();
692         if (isEnd) {
693             if (buffer_len != 0) {
694                 switch (currentGroup) {
695                     case gComment:
696                         lexemTable.push_back(new Lexem(buffer, Comment));
697                         buffer.clear();
698                         break;
699                     case gMultiComment:
700                         errorTable.push_back(new Error(currentRow, currentPos,
701                             buffer, L"Multirow_comment_is_not_finished"));
702                         buffer.clear();
703                         break;
704                     case gVar:
705                         errorTable.push_back(new Error(currentRow, currentPos,
706                             buffer, L"Value_of_Var_is_not_finished"));
707                         buffer.clear();
708                         break;
709                 }
710             }
711             else {
712                 switch (currentGroup) {
713                     case gComment:
714                         y_Comment(L' ');
715                         buffer.clear();
716                         break;
717                     case gMultiComment:
718                         errorTable.push_back(new Error(currentRow, currentPos,
719                             buffer, L"Multirow_comment_is_not_finished"));
720                         buffer.clear();
721                         break;
722                 }
723             }
724             else {
725                 switch (currentGroup) {
726                     case gNone:
727                         errorTable.push_back(new Error(currentRow, currentPos - 1,
728                             buffer, L"Unexpected_symbol"));
729                         fsm->popFront();
730
731                         buffer.clear();

```

```

729         fsm->setState(0);
730         break;
731     case gComment:
732         errorTable.push_back(new Error(currentRow, currentPos, L"",
733                                     L"Comment is not finished"));
734         fsm->setState(0);
735         break;
736     case gId:
737         errorTable.push_back(new Error(currentRow, currentPos,
738                                     buffer, L"Unexpected symbol in identifier"));
739         buffer.clear();
740         fsm->setState(0);
741         break;
742     case gVar:
743         errorTable.push_back(new Error(currentRow, currentPos,
744                                     buffer, L"Unexpected symbol in variable"));
745         while (is_digit(fsm->front()) or is_letter(fsm->front())) {
746             buffer.push_back(fsm->front());
747             fsm->popFront();
748             pos_delta++;
749         }
750         buffer.clear();
751
752         fsm->setState(0);
753         break;
754     case gWord:
755         currentGroup = gId;
756         fsm->setState(12);
757         break;
758     };
759 };
760 };
761 fsm->assignTransitions(200, tM, errorProcessor);
762 return fsm;
763 }

```